

학사학위 청구논문

StreamAlert:
경량 AI 기반 실시간 방송 재미도
분석 및 개인화 알림 시스템

(StreamAlert: A Lightweight AI-Based Real-Time
Broadcasting Fun-Score Analysis and Personalized
Alert System)

2025년12월04일

숭실대학교 IT대학

AI융합학부

김 성 재

학사학위 청구논문

StreamAlert:
경량 AI 기반 실시간 방송 재미도
분석 및 개인화 알림 시스템

StreamAlert: A Lightweight AI-Based Real-Time
Broadcasting Fun-Score Analysis and Personalized
Alert System

지도교수 : 박 건 우

이 논문을 학사학위 논문으로 제출함

2025년12월04일

숭실대학교 IT대학

AI융합학부

김 성 재

김성재의 학사학위 논문을 인준함

심사위원장 김 강 희 (인)

심 사 위 원 박 건 우 (인)

2025년 12월 04일

숭실대학교 IT대학

감사의 글

본 논문이 완성될 수 있도록 지도해 주신 박건우 교수님께 깊은 감사의 말씀을 드립니다. 또한 학업과 연구를 응원해 준 가족과 친구들에게도 감사드립니다.

목 차

표 및 그림 목차	i
국문초록	ii
I. 서론	1
II. 관련 연구 및 기초 이론	3
II-1. 1인가구와 여가·콘텐츠 소비 연구	3
II-2. 인터넷 방송 알림 시스템의 발전.....	4
II-3. 감정 분석 및 재미도 측정 기법.....	4
II-4. 채팅 기반 하이라이트 추출 연구	4
II-5. 본 연구의 차별성	5
III. StreamAlert 시스템 설계 및 구현	6
III-1. 시스템 아키텍처 개요	6
III-2. 기술스택	7
III-3. 기능별 상세 설계	9
III-4. 데이터 수집 알고리즘	12
III-5 재미도 분석 알고리즘	13
III-6. 하이라이트 판단 알고리즘	16
III-7. 핫클립 분석 알고리즘	17
III-8. 최적화 기법	17
IV. 모의실험 결과 및 토의	18
IV-1. 실험 환경	18
IV-2. 시스템 성능 결과	18
IV-3. 재미도 분석 성과	21
IV-4. 타 기법과 비교	27
IV-5. 실험 결과 종합	29
V. 결론	31
참고문헌	33
부록	35

표 및 그림 목차

표 1.1 기술 스택	7
표 1.2 핵심 재미도 분석 알고리즘 구성	8
표 1.3 외부 서비스 및 API	8
표 1.4 클라우드 인프라 및 데이터베이스	8
표 1.5 보조 도구 및 라이브러리	9
표 2.1 플랫폼별 API 호출 성능	19
표 2.2 알림 채널별 전송 성능	19
표 2.3 비교 분석 결과	23
표 3 재미도 분석 기법 비교	27
그림 1 시스템 아키텍처	6
그림 2.1 CPU 활용률	20
그림 2.2 메모리 활용률	20
그림 3 재미도 점수 변화	21
그림 4.1 재미도 점수 통계	23
그림 4.1 하이라이트 점수 통계	23
그림 5.1 재미도 점수 Q-Q plot	25
그림 5.2 하이라이트 점수 Q-Q plot	25

StreamAlert: 경량 AI 기반 실시간 방송 재미도 분석 및 개인화 알림 시스템

AI융합학부 김 성 재

지도교수 박 건 우

본 연구는 인터넷 방송 환경에서 시청자와 크리에이터 간의 상호작용 데이터를 분석하여 실시간으로 재미도(Fun Score)를 계산하고, 이를 기반으로 사용자 맞춤형 알림을 제공하는 시스템 StreamAlert를 제안한다. 제안 시스템은 Chzzk, Soop, YouTube, 네이버 카페 등 다양한 플랫폼의 활동 정보를 API 연동 및 웹 크롤링을 통해 통합적으로 수집하며, 방송 시작/종료, 방송 제목 변경, 생성된 핫클립, VOD 업로드, 업로드된 유튜브 영상, 팬카페 게시물 여부 등을 탐지하여 사용자에게 즉각적인 알림을 전송한다.

또한, 저사양 서버 환경에서도 구동 가능한 경량 AI 기반 통계적 분석 방법을 적용하여 방송 중 하이라이트 순간을 실시간으로 탐지하고, Discord 및 웹 애플리케이션을 통해 사용자에게 전달한다. 주요 재미도 분석 알고리즘은 채팅 급증, 반응 키워드, 참여 다양성, 시청자 수 변화를 종합하여 정량적 점수를 산출하며, 이 점수를 바탕으로 자동적인 하이라이트 검출이 가능하다. 실제 서비스 환경에서의 실험 결과 평균 알림 지연 시간은 1000ms 이하였으며, 플랫폼별 API 호출 성공률은 99.99%를 달성하였다. 추가적으로 규칙 기반 탐지 및 딥러닝 기반 감정 분석 기법과의 비교 실험을 통해 제안 방법이 정량성과 실시간성을 동시에 만족함을 확인하였다.

I. 서 론

최근 인터넷 방송 시장은 플랫폼의 다양화와 함께 폭발적인 성장을 보이고 있다. 유튜브(YouTube), 치지직(Chzzk), 아프리카TV, 숲(Soop) 등 여러 플랫폼이 등장하며 시청자들은 다양한 콘텐츠를 실시간으로 소비하게 되었다. 이러한 변화는 1인가구, 특히 청년 1인가구의 여가시간 활용 방식과도 밀접한 관련이 있다.

통계청의 「2014년 생활시간조사 결과」에 따르면, 1인가구의 비중은 꾸준히 증가하고 있으며, 20~30대 청년층을 중심으로 하루 평균 여가시간 중 온라인 미디어 소비 비중이 40% 이상으로 나타났다. 이는 여가시간의 주요 소비 형태가 오프라인 활동에서 디지털 콘텐츠 중심으로 전환되고 있음을 보여준다. 송혜림 등[1]은 청년 1인가구의 여가 및 식사활동을 분석하며, 성별과 취업 여부에 따라 여가활동의 질과 시간 배분이 달라진다고 밝혔다. 특히 청년층은 실시간 방송, 유튜브 시청, SNS 활동 등을 통해 비대면 방식으로 여가를 즐기며, 이러한 형태의 여가활동이 심리적 안정감과 만족도 향상에 기여한다고 보고했다.

서지원과 양진운[2] 역시 청년 1인가구의 생활시간 사용구조를 분석하면서, 남성은 방송 시청과 게임 중심, 여성은 모바일 콘텐츠 소비 중심의 여가활동을 보인다고 제시했다. 이는 1인가구의 여가 형태가 성별·직업적 특성에 따라 차별화되고 있음을 시사한다.

또한 우민희 등[3]은 청년세대 1인가구가 다인가구에 비해 여가활동에서 즉흥성과 개인화 성향이 강하다고 지적했다. 이들은 ‘혼자 하는 여가(솔로 레저)’가 스트리밍 방송, 온라인 콘텐츠 시청 등 디지털 기반 실시간 활동 중심으로 변화하고 있음을 강조했다.

이러한 사회적 변화는 방송 플랫폼의 기술적 발전과 맞물려, 개별 시청자의 취향과 시청 습관에 따라 맞춤형 알림 및 추천이 가능해야 함을 보여준다. 따라서 본 연구에서는 실시간 방송 중 시청자의 반응 데이터를 분석하여 ‘재미도(Fun Score)’를 계산하고, 이를 기반으로 한 개인화 알림 시스템 StreamAlert를 제안한다.

본 연구는 저사양 서버에서도 작동 가능한 경량 AI 분석 모델을 통해 실시간으로 채팅, 반응량, 시청자 수 등을 측정하고 방송의 하이라이트 순간을 자동 탐지하여 사용자에게 맞춤 알림을 제공한다. 이를 통해 방송 시청의 몰입도 향상과, 다양한 플랫폼의 정보 접근성을 개선하고자 한다.

II. 관련 연구 및 기초 이론

II-1. 1인가구와 여가·콘텐츠 소비 연구

국내 연구에 따르면 1인가구는 여가시간의 상당 부분을 디지털 콘텐츠 소비에 사용하며, 이러한 경향은 청년층일수록 더욱 두드러진다.

송혜림·강은주·김민주(2018)는 청년 1인가구의 생활시간 데이터를 분석하여, 남성 1인가구는 방송 및 게임 중심의 여가활동을, 여성 1인가구는 SNS·스트리밍 시청 중심의 활동을 선호한다고 밝혔다. 이 연구는 1인가구의 여가시간이 비대면 콘텐츠 소비 중심으로 재편되고 있음을 보여준다.

서지원과 양진운[2]은 청년 1인가구의 성별 생활시간 구조를 비교 분석하며, 여성의 경우 ‘시간 빈곤감’을 더 크게 느끼지만 짧은 여가시간 내에서도 SNS와 실시간 방송 소비를 통해 사회적 연결감을 유지하려는 경향이 있다고 제시했다. 즉, 청년 1인가구의 여가활동은 단순한 시간 소비를 넘어 심리적 안정과 사회적 소속감을 형성하는 중요한 수단으로 작용한다.

우민희 등[3]의 연구에서는 청년세대 1인가구가 여가활동에서 개인 선택의 자율성을 중시하며, 자신의 흥미에 따라 즉흥적으로 콘텐츠를 소비하는 경향이 강하다고 분석했다. 특히 실시간 방송이나 온라인 스트리밍 시청은 이들의 ‘혼자 있는 시간’을 능동적 참여형 여가로 전환시키는 중요한 매체로 나타났다.

한편, 송완구 등[4]은 관광·여행 영역에서 1인가구의 여가활동이 단체 중심에서 개별화된 행동으로 전환되고 있음을 밝혔다. 이 연구는 디지털 기반의 여가 활동이 오프라인 참여형 여가로 확장될 가능성을 보여준다.

이처럼 선행연구들은 청년 1인가구의 여가활동이 ‘온라인 실시간 콘텐츠 중심’으로 변화하고 있음을 공통적으로 지적한다. 따라서 본 연구는 이러한 사회문화적 맥락을 반영하여 실시간 방송 중 시청자의 반응 데이터를 분석하고, 개인 맞춤형 알림을 제공하는 StreamAlert 시스템을 설계·구현한다.

II-2. 인터넷 방송 알림 시스템의 발전

기존 인터넷 방송 플랫폼은 자체 구독/팔로우 기반 알림 시스템을 제공하지만 이는 각 플랫폼 내부에 국한되어 있다. 최근 Discord, 카카오톡 봇 등의 연계 서비스가 부분적으로 등장하였지만, 이들 역시 방송의 시간적 이벤트만 알림할 뿐 방송 내용의 맥락적 분석(재미도, 감정 반응, 방송 제목 변경, VOD 업로드 등)을 제공하지 못하는 한계가 있다.

II-3. 감정 분석 및 재미도 측정 기법

자연어 처리 분야에서는 시청자 메시지(채팅)를 활용한 감정 분석이 활발히 연구되고 있다. 관련 선행연구는 크게 규칙 기반 접근과 딥러닝 기반 접근으로 구분된다. 규칙 기반 접근(특정 키워드("ㅋㅋㅋ", "ㅠㅠ", "헐")나 이모티콘을 감정 카테고리로 단순 매핑. → 계산량이 적어 실시간 분석 가능하나, 은유적 표현이나 신조어 반영에는 한계. 딥러닝 기반 접근(KoBERT, Transformer 등)은 문맥을 고려한 정밀한 감정 분류가 가능하나, 자원 소모가 커서 실시간 환경(특히 저사양 서버)에는 부적합하다. 본 연구는 이들 두 접근법의 중간 지점에서, 경량 AI + 통계 기반 분석을 통해 키워드 매칭, 채팅량 급증, 사용자 다양성, 시청자 증감을 종합한 정량적 재미도(Fun Score)를 계산한다.

II-4. 채팅 기반 하이라이트 추출 연구

최근에는 실시간 방송 중 채팅 반응을 방송 하이라이트 탐지 지표로 활용하는 연구가 활발히 진행되고 있다.

이주현과 염익준[6]은 Twitch TV의 실시간 채팅 데이터를 활용하여 하이라이트 구간을 자동 추출하는 방법을 제안하였다. 연구에서는 KoNLPy 기반 감정 분석을 이용해 채팅 메시지의 내용적 반응을 수치화하고, 특정 시점에서의 반응 급증 구간을 하이라이트로 탐지하였다.

문하겸[7]은 사용자 채팅 데이터를 기반으로 스트리밍 방송에서 하이라이트 구간을 자동 탐지하는 시스템을 개발하였으며, 채팅량·감정어 빈도·시청자 반응 패턴 등을 결합하여 방송의 주요 순간을 실시간으로 판별하였다.

김은율과 이계민[8]은 채팅 트래픽 분석을 통해 개인방송 하이라이트를 검출하는 알고리즘을 제시하였으며, 게임 콘텐츠를 대상으로 실험을 수행하여 채팅 반응 급증이 실제 하이라이트 장면과 높은 상관관계를 가짐을 검증하였다. 또한 김은율[9]은 딥러닝 기반의 양방향 소통 영상 분석을 통해 시청자 반응과 방송 맥락을 동시에 고려한 하이라이트 검출 모델을 제안하였다. 해당 연구는 채팅 데이터 외에도 영상 프레임의 특징을 학습하여 복합적인 반응을 탐지할 수 있음을 보여준다.

이들 연구는 공통적으로 실시간 채팅 데이터를 하이라이트 탐지의 핵심 지표로 활용하고 있으나, 대부분 단일 플랫폼 또는 단일 반응 변수(예: 채팅량, 감정어 빈도 등)에 의존하는 한계를 갖는다. 본 연구는 이러한 선행연구의 한계를 보완하여 채팅 급증, 감정 반응, 참여 다양성, 시청자 추세를 통합한 정량적 모델을 제시하고, 실시간성 및 경량성을 동시에 만족하는 알고리즘으로 확장하였다.

II-5. 본 연구의 차별성

정리하면, 기존 연구와 본 논문의 차별성은 다음과 같다.

멀티플랫폼 통합 알림: 단일 플랫폼 알림의 한계를 극복하고, Chzzk, Soop, YouTube, 네이버 카페 정보를 통합 분석하여 사용자에게 일원화된 알림 서비스를 제공한다.

경량 AI 기반 정량 모델: Fun Score를 수식 기반의 정량 지표로 제시하여, 단순 "하이라이트 구간" 탐지에서 벗어나 비교·재현·개선 가능한 연구 자산을 생성한다.

실시간성 보장: 저사양 환경(1GB RAM)에서도 안정적으로 실시간 처리 가능함을 실험적으로 입증하여, 개인 개발자 및 소규모 서비스에서도 활용 가능하다.

III. StreamAlert 시스템 설계 및 구현

III-1. 시스템 아키텍처 개요

본 시스템은 [그림 1]과 같은 흐름으로 작동한다.

StreamAlert는 다음 7가지 핵심 기능을 제공하는 통합 알림 시스템이다.

방송 상태 알림: 방송 시작(뱅온), 종료(뱅종), 방제 변경

실시간 하이라이트 알림: 재미도 분석 기반 하이라이트 순간 탐지 및 알림

핫클립 알림: 조회수 급증 클립 자동 탐지

VOD 업로드 알림: 방송의 다시보기 영상 업로드 감지

유튜브 업로드 알림: YouTube 영상 업로드 감지

팬카페 알림: 네이버 카페 새 게시물 알림

VOD 하이라이트 댓글 작성: 하이라이트 타임라인 댓글을 VOD에 작성

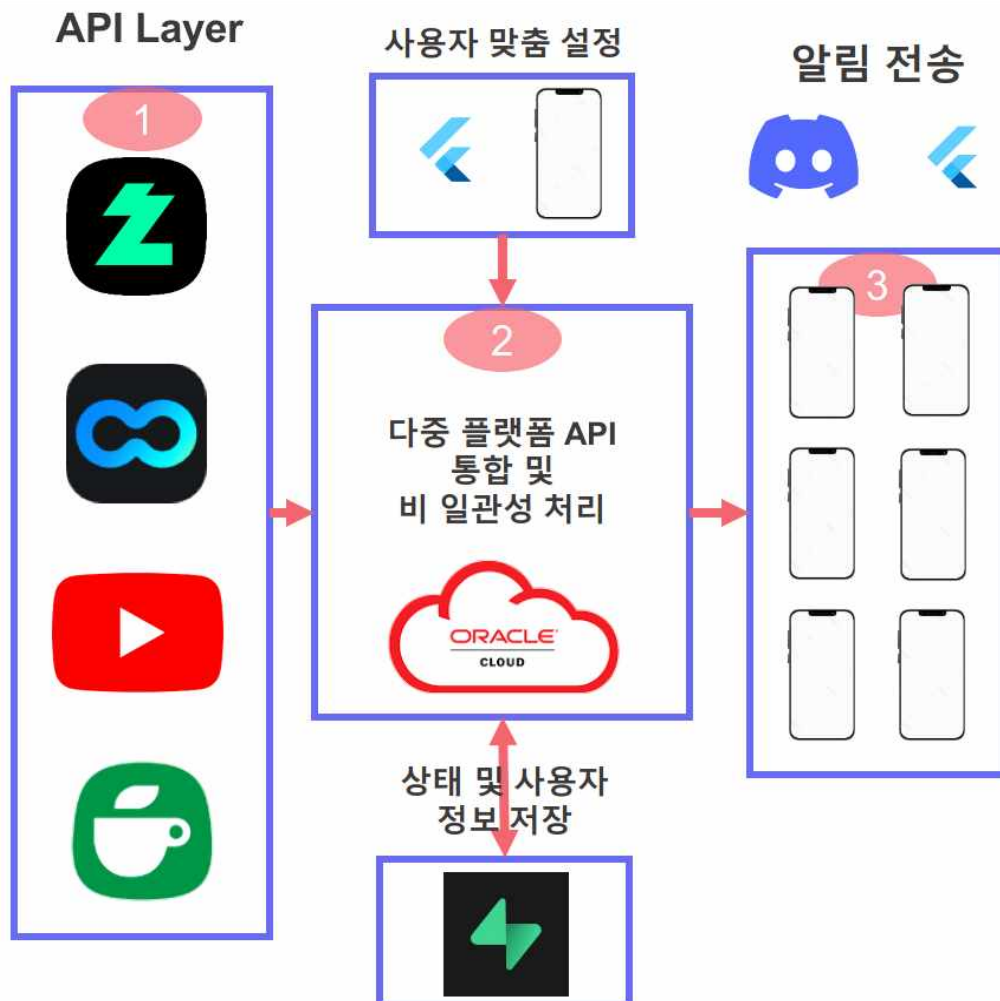


그림 1 시스템 아키텍처

1. 데이터 수집 계층:

Chzzk API, Sopo API, YouTube Data API, Naver Cafe API를 통해 실시간 데이터 수집.

플랫폼별 방송 상태, 채팅, 시청자 수, VOD, 클립, 네이버 카페 게시글 정보, 유튜브 영상 데이터 통합 수집

2. 데이터 처리 계층:

비동기 FastAPI 서버 기반으로 방송 상태, 채팅, 시청자 수 등 데이터 처리

방송 상태 모니터링: 온라인/오프라인 전환, 방제 변경 감지(2초 간격)

재미도 분석: 채팅 데이터 기반 실시간 재미도 계산 (5초 간격, 30초 윈도우)

Description 생성: 하이라이트 데이터로 gemini API를 사용하여 생성

핫클립 분석: 조회수 및 최신성 기반 점수 계산 (30분 간격)

콘텐츠 모니터링: VOD 업로드, 유튜브 영상 업로드, 카페 게시글 탐지(3초 간격)

2-1. 채팅 로그를 윈도우 단위로 나누어 재미도 점수를 실시간 계산

3. 알림 전송 계층:

Discord Webhook, Firebase FCM(Flutter 앱) 통해 사용자에게 알림 전송.

사용자별 맞춤 알림 설정 (방송/하이라이트/핫클립/VOD/유튜브/카페 선택 가능)

III-2. 기술 스택

분야	기술/언어	역할 및 용도
프론트엔드 기능	Flutter, Dart	크로스플랫폼 웹앱 개발
백엔드 기능	Python 3.13.3, FastAPI	서버 로직, API 개발 및 비동기 처리
데이터 분석	Pandas, Stats, re, genai	정규표현식,패턴 매칭, 재미도 계산, 하이라이트 설명 생성
데이터 처리	Pandas, Asyncio, JSON, Deque, PILImage	데이터 변환, 분석 및 비동기 작업 처리, 이미지 추출
웹 통신	Aiohttp, requests, websockets	HTTP 요청, 웹 소켓 통신
데이터 검증	Dataclasses, tencity	데이터 구조 정의 및 재시도 로직

표 1.1.개발 기술 스택

구성 요소	가중치	계산 방식	설명
채팅 급증	45%	시그모이드 변환 기반 정규화	각 방송 채팅 수 대비 상대적 증 가율
감정 반응	30%	키워드 가중치 x 밀도	웃음(4.0), 흥분 (3.5), 놀람(2.5), 일 반반응(1.0)
알림 서비스	10%	작성자/ 길이/시 간 분산도	고유 작성자 수, 메시지 길이, 메 시지 작성 시간 표준편차
시청자 수	15%	단기/즉시/지속적 증가 패턴	20분, 1분, 연속 증가 패턴 분석

표 1.2. 핵심 재미도 분석 알고리즘 구성

서비스 분류	서비스명	활용 목적
스트리밍 플랫폼	Chzzk API, Soop API	실시간 방송 정보 수집
콘텐츠 플랫폼	YouTube Data API v3, Naver Cafe AP	영상 업로드 정보 수집, 카페 게시글 모니터링
알림 서비스	Discord Webhook API, Firebase FCM	실시간 알림 전송

표 1.3. 외부 서비스 및 API

분류	서비스	용도
데이터 베이스	Supabase, Oracle cloud	사용자 데이터, 설정 정 보 저장
클라우드 호스팅	Oracle Cloud, Firebase Hosting	서버 배포 및 웹앱 호 스팅
이미지 저장	Imgbb API	썸네일 이미지 업로드 및 저장

표 1.4. 클라우드 인프라 및 데이터베이스

분류	도구/라이브러리	사용 목적
스케줄링	APScheduler	주기적 작업 및 통계 계산
SSL/보안	Certify, ssl	안전한 웹소켓 연결
개발 도구	Python-dotenv, urllib	환경 설정 및 URL 처리

표 1.5.보조 도구 및 라이브러리

III-3. 기능별 상세 설계

III-3-1. 방송 상태 알림 (live_message.py)

모니터링 항목:

- 방송 시작(뱅온): CLOSE → OPEN 상태 전환 감지
- 방송 종료(뱅종): OPEN → CLOSE 상태 전환 감지
- 방제 변경: 방송 중 제목 변경 감지

처리 프로세스:

1. 플랫폼 API에서 방송 상태 데이터 수집 (_get_state_data())
2. 응답 데이터 유효성 검증 (_is_valid_state_data())
3. 방송 제목, 시청자 수, 썸네일 등 정보 추출 및 업데이트 (_update_stream_info())
4. 온라인 상태 처리: 뱅온 또는 방제 변경 감지 시 알림 생성 (_handle_online_status())
5. 오프라인 상태 처리: 뱅종 감지 시 알림 생성 (_handle_offline_status())

알림 생성 조건:

- 뱅온: live_state == "OPEN" AND 마지막 업데이트 시간 < 방송 시작 시간
- 뱅종: live_state == "CLOSE" AND 마지막 업데이트 시간 < 방송 종료 시간
- 방제 변경: 현재 제목 != 이전 제목 AND 방송 중

상세 구현 코드는 부록 1절을 참조한다.

III-3-2. 실시간 하이라이트 알림 (chat_analyzer.py)

재미도 분석 기반 하이라이트 탐지 (III-5절, III-6절 참조)

분석 주기:

- 5초마다 재미도 점수 계산
- 30초 윈도우 단위로 채팅 데이터 분석

하이라이트 판단 기준:

- 작은 하이라이트 ($\text{score_difference} \geq 15$): VOD 타임라인 댓글로 저장
- 큰 하이라이트 ($\text{score_difference} \geq 70$): 실시간 알림 전송

알림 전송 프로세스:

1. 하이라이트 검출: $\text{score_difference} \geq 70$ 시 큰 하이라이트로 분류
2. 썸네일 캡처: Imgbb API를 통해 실시간 방송 화면 업로드
3. AI 설명 생성: Google Gemini API로 하이라이트 순간 자연어 설명 생성
4. 사용자별 필터링 후 Discord Webhook 및 Firebase FCM으로 알림 전송
5. 모든 하이라이트 데이터를 JSON 파일로 저장 (VOD 댓글 작성용)

상세 구현 코드는 부록 2절을 참조한다.

III-3-3. 핫클립 알림 (unified_hot_clip.py)

하이라이트 분석 알고리즘 (III-7절 참조)

분석 주기:

- 30분마다 최근 14일간 클립 데이터 수집

핫클립 판단 알고리즘:

1. 클립 데이터 수집: 플랫폼 API에서 최근 14일 내 생성된 모든 클립 조회
 2. 기준 메트릭 계산: 평균 조회수 (AvgViews) 산출
 3. 핫클립 점수 계산:
 - 최종 점수: $\text{HotScore} = \text{조회수 점수 (70\%)} + \text{최신성 점수 (30\%)}$
 4. 핫클립 판단: $\text{HotScore} \geq 70$ 인 클립을 핫클립으로 분류
 5. 중복 방지: 이미 알림 보낸 클립은 데이터베이스에서 제외
 6. 알림 전송: 사용자별 설정에 따라 Discord 및 FCM으로 알림 전송
- 상세 구현 코드는 부록 3절을 참조한다.

III-3-4. VOD 업로드 알림 (unified_vod.py)

처리 프로세스:

1. API에서 VOD 데이터 확인 (check_video())
2. 새 VOD 여부 판단: 기존 VOD 목록과 비교
3. 새 VOD 발견 시 JSON 데이터 생성 및 저장
4. VOD 알림 전송 (post_video())

상세 구현 코드는 부록 4.1~4.3절을 참조한다.

III-3-5. 유튜브 업로드 알림 (getYoutubeJsonData.py)

처리 프로세스:

1. YouTube Data API로 채널 정보 조회
2. 현재 영상 개수와 저장된 영상 개수 비교
3. 새 영상 발견 시 Search API로 최신 영상 정보 수집
4. 영상 데이터 JSON 생성 및 알림 전송
5. 데이터베이스 업데이트

상세 구현 코드는 부록 5절을 참조한다.

III-3-6. 팬카페 게시글 알림 (getCafePostTitle.py)

처리 프로세스:

1. 네이버 카페 API에서 게시글 목록 조회
2. 특정 작성자(스트리머) 필터링
3. 새 게시글 판단: 게시글 ID 및 작성 시간 비교
4. 새 게시글 발견 시 알림 데이터 생성 및 전송
5. 처리된 게시글 ID 저장 (최대 10개 유지)

상세 구현 코드는 부록 6절을 참조한다.

III-3-7. 하이라이트 타임라인 댓글 (unified_vod.py)

처리 프로세스:

1. VOD 업로드 감지 시 하이라이트 데이터 대기 후 하이라이트 파일 검색
1. 방송 제목 및 영상길이를 VOD-하이라이트 매칭

2. 하이라이트 시간을 VOD 타임라인으로 변환 (세그먼트 오프셋 적용)
3. 재미도 점수 기반 댓글 포맷 생성
4. VOD에 타임라인 댓글 작성

상세 구현 코드는 부록 4.4~4.7절을 참조한다.

III-3-8. 통합 알림 관리 (discord_webhook_sender.py)

사용자별 알림 설정:

- Supabase 데이터베이스에 사용자별 알림 선호도 설정 저장
- 알림 유형별 활성화/비활성화 설정 (뱅은, 방제 변경, 하이라이트, 핫클립, VOD, 유튜브, 카페)
- 채널별 세부 설정 지원

알림 필터링:

- get_list_of_urls() 함수로 사용자 설정 기반 전송 대상 필터링
- 알림 유형과 채널 ID를 조건으로 Discord Webhook URL 목록 반환

알림 전송:

- Discord Webhook 및 Firebase FCM 동시 전송
- 세마포어 기반 동시 요청 제한 (최대 10개)
- 실패 시 최대 3회 재시도 (지수적 백오프)

상세 구현 코드는 부록 7~8절을 참조한다.

III-4. 데이터 수집 알고리즘

본 연구는 다양한 플랫폼(Chzzk, Soop, YouTube, naver cafe, imgbb 등)의 공개 API 및 웹 응답을 통합하여 수집하기 위한 데이터 수집 모듈을 구현하였다. 각 플랫폼은 요청에 필요한 쿠키·파라미터·헤더 정보 및 응답 형식이 상이하므로, 이를 통합적으로 관리할 수 있도록 플랫폼별 설정(configuration)을 사전 정의한 후 단일 함수에서 공통 절차에 따라 요청을 수행하도록 설계하였다.

1. 플랫폼별 환경 설정: 각 플랫폼의 요청 요건(쿠키, 파라미터, 헤더 포함 여부)을 확인하고 설정값을 불러온다.
2. 요청 실행: 비동기(async) 방식으로 HTTP 요청을 수행하여 API 응답을 획득한다.

3. 응답 검증: 서버로부터 정상 응답(status code 200)이 도착한 경우 응답을 파싱하여 결과를 반환한다.
 4. 오류 처리 및 재시도: 서버 오류(5xx)의 경우 지수적 backoff(exponential backoff) 기반으로 최대 3회 재시도한다. 클라이언트 오류(4xx)의 경우 추가 시도 없이 종료한다. 연결 오류(ConnectionError, Timeout, SSL 오류 등)에 대해서는 별도의 예외 처리와 대기 시간을 적용하여 안정성을 강화하였다. 5. 성능 로깅: 모든 시도에 대해 응답 시간, 재시도 횟수, 오류 유형을 기록하여 추후 데이터 수집 효율성을 분석할 수 있도록 하였다.
- 본 모듈을 활용하여 본 연구는 신뢰성 있는 대규모 데이터 수집을 가능하게 하였다. 특히 플랫폼별로 상이한 요청 환경을 단일 구조로 통합하고, 지수적 backoff 기반의 재시도 메커니즘을 적용함으로써 수집 과정의 안정성과 효율성을 크게 향상시켰다. 또한 수집 성능 로그를 기반으로 데이터 수집 인프라의 병목 현상 및 실패 원인을 추적할 수 있었다.
- 상세 구현 코드는 부록 9절을 참조한다.

III-5. 재시도 분석 알고리즘

본 연구는 경량 AI 모델을 기반으로 다음 네 가지 지표를 합산하여 FunScore를 계산하였다.

기준 평균 값 업데이트

$$Mavg^{(t)} = \alpha * Mavg^{(t)} + (1 - \alpha) * Mavg^{(t-1)} \quad (1)$$

$$Vavg^{(t)} = \alpha * Vavg^{(t)} + (1 - \alpha) * Vavg^{(t-1)} \quad (2)$$

$$Tavg^{(t)} = \alpha * Tavg^{(t)} + (1 - \alpha) * Tavg^{(t-1)} \quad (3)$$

$Mavg$: 기존의 채팅 수
 $Vavg$: 기존의 시청자 수
 $Tavg$: 기존의 임계 값

α : 최근 20분(240회)의 데이터가 90% 반영되도록 [$\alpha = 1 - 0.1^{\left(\frac{1}{240}\right)} \approx 0.0958$]

1. 채팅 급증 점수 (45%)

$$Score_{chat} = \min\left(\sigma\left(\left(\frac{Mwin}{Mavg} - 3.0\right) * 100\right), 100\right) \quad (4)$$

$Mchat$: 현재 윈도우 채팅 수

$$\sigma(x, m): \text{시그모이드 변환 함수} = \frac{2}{1 + e^{-(x-m)}}$$

2. 반응 강도 점수 (30%)

특정 키워드를 감정 강도별로 가중치를 합산하여 점수를 산출한다. 단순히 키워드 등장 횟수만을 고려하지 않고, 본 연구에서는 반응 문자열 길이에 따른 구간별 가중치를 적용하였다. 긴 문자열은 더 강한 반응 신호를 의미하므로 추가 점수를 부여한다.

웃음(laugh): (1 - 5자: 1.0점), (6 - 15자: 1.2점), (16 - 30자: 1.5점), (31 - 50자: 1.8점), (51자 이상: 2.0점)

흥분(excitement): (1 - 5자: 1.0점), (6 - 10자: 1.2점), (11 - 20자: 1.4점), (21자 이상: 1.6점)

놀람(surprise): (모든 길이: 1.0점)

일반 반응(reaction): (1 - 5자: 1.0점), (6 - 10자: 1.1점), (11 - 20자: 1.3점), (21자 이상: 1.5점)

인사(greeting): (모든 길이: 1.0점)

$$Score_{reaction} = \min\left(0, \frac{\sum_k w_k L_k C_k}{Mchat}, 9.0\right) * 100, 100 \quad (5)$$

C_k : 감정 키워드 등장 횟수

L_k : 해당 키워드 길이에 따른 길이 가중치 (위 표 참고)

w_k : 키워드 가중치 (웃음=4.0, 흥분=3.5, 놀람=2.5, 일반반응=1.0)

3. 참여 다양성 점수 (10%)

고유 시청자 수 비율, 메시지 길이 분산, 시간 간격 표준편차 반영.

$$Score_{Diversity} = UserDiv + LengthDiv + TimeDiv \quad (6)$$

사용자 다양성:

$$UserDiv = \min\left(\frac{U}{M} * 50, 50\right) \quad (7)$$

U: 고유 사용자 수

M: 전체 메시지 수

메시지 길이 다양성:

$$LengthDiv = \min\left(\frac{stdev(L)}{20}, 10\right) \quad (8)$$

L: 메시지 길이 집합

시간 간격 다양성:

$$TimeDiv = \min\left(\frac{stdev(\Delta T)}{5}, 40\right) \quad (9)$$

ΔT : 인접메시지 간격

4. 시청자 추세 점수(15%)

$$Score_{viewer} = \min(s1 + s2 + s3, 100) \quad (10)$$

$$s1 = \left\{ \begin{array}{l} \text{단기 증가 추세} \\ 40 \text{ if } \frac{V_{recent10}}{V_{prev10}} \geq 1.30 \\ 30 \text{ if } 1.20 \leq \frac{V_{recent10}}{V_{prev10}} < 1.30 \\ 20 \text{ if } 1.10 \leq \frac{V_{recent10}}{V_{prev10}} < 1.20 \\ 10 \text{ if } 1.05 \leq \frac{V_{recent10}}{V_{prev10}} < 1.10 \\ 0 \text{ otherwise} \end{array} \right. \quad (11)$$

$$s2 = \left\{ \begin{array}{l} \text{즉시 급등} \\ 30 \text{ if } \frac{V_{current}}{V_{reconf1}} \geq 1.50 \\ 20 \text{ if } 1.30 \leq \frac{V_{current}}{V_{reconf1}} < 1.50 \\ 15 \text{ if } 1.20 \leq \frac{V_{current}}{V_{reconf1}} < 1.30 \\ 10 \text{ if } 1.10 \leq \frac{V_{current}}{V_{reconf1}} < 1.20 \\ 0 \text{ otherwise} \end{array} \right. \quad (12)$$

$$B_i = \frac{1}{|I_i|} \sum_{t \in I_i} V(t), i = 1, 2, 3, 4, 5$$

$V(t)$ = 시각 t 의 시청자수

I_i = 최근 20분을 균등하게 5분할 한 구간 (13)

$$Increasing_count = \sum_{i=2}^5 1(B_i > B_{i-1}) \quad (14)$$

$$S_3 = \begin{cases} 30 & \text{if } Increasing_count \geq 4 \\ 20 & \text{if } Increasing_count = 3 \\ 10 & \text{if } Increasing_count = 2 \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

5. 최종 Fun Score

$$FunScore = 0.45 * Score_{chat} + 0.30 * Score_{reaction} + 0.10 * Score_{Diversity} + 0.15 * Score_{viewer}$$

해당 FunScore를 5초 마다 분석을 하고, 최근 30초 동안의 채팅 데이터가 window가 된다.

III-6. 하이라이트 판단 알고리즘

본 연구는 계산된 재미도(Fun Score)를 기반으로 방송 중 의미 있는 순간(하이라이트)을 자동으로 판별한다. 하이라이트 판단 알고리즘은 단순히 점수가 높은 시점을 선택하는 것이 아니라, 다음과 같은 조건을 종합적으로 고려한다.

1. 기준점 대비 점수 상승(difference score): 방송 환경의 다양성을 고려하여, 직전 1분 구간의 최소 재미도 점수 대비 현재 점수의 차이를 계산한다.

$$D_t = \max(F_t - \min_{i \in W}(F_i), 0) \quad (17)$$

여기서 F_t 는 시점 t의 최종 Fun Score, W는 직전 1분 구간이다.

$$\text{하이라이트 분류} \begin{cases} \text{작은 하이라이트} & \text{if } D_t \geq T_{small} = 15 \\ \text{큰 하이라이트} & \text{if } D_t \geq T_{big} = 70 \end{cases}$$

2. 연속 하이라이트 병합

시간적으로 인접한 하이라이트는 단일 이벤트로 병합된다.

병합 조건은 다음과 같다.

병합조건: $\Delta t < 120\text{sec} \wedge N_{gap} < 3$

- Δt : 현재 하이라이트와 직전 하이라이트 간 시간 간격
- N_{gap} : 두 하이라이트 사이의 비-하이라이트 구간 개수

병합 시, 최종 하이라이트 정보는 다음과 같이 결정된다.

$$D_{final} = \max(D_{prev}, D_{current})^{t_{final} = t_{first}} \quad (18)$$

3. 알림 전송 기준

작은 하이라이트 ($D_t \geq 15$): VOD 타임라인 댓글로 저장

큰 하이라이트 ($D_t \geq 70$): 실시간 알림 전송 + VOD 댓글 저장

상세 구현 코드는 부록 10절을 참조한다.

III-7. 핫클립 분석 알고리즘

본 연구에서는 인터넷 방송 플랫폼(Chzzk, Soop 등)에서 제공하는 클립 데이터를 기반으로, 최근 일정 기간 내에 생성된 클립을 분석하여 “핫클립 (Hot Clip)”을 자동 탐지하는 알고리즘을 설계하였다.

1. 조회수 기반 점수 계산

$$ViewScore = \min\left(\sigma\left(\frac{ReadCount}{AvgViews}, 3.0\right) * 100, 100\right) \quad (19)$$

$ReadCount$ 는 조회수
 $AvgViews$ 최근14일간 모든 클립의 평균 조회수

2. 최신성 기반 점수 계산

$$ReccencyScore(h) = \begin{cases} 100, h \leq 1 \\ 90, 1 < h \leq 6 \\ 70, 6 < h \leq 24 \\ 50, 24 < h \leq 72 \\ 30, 72 < h \leq 168 \\ 10, h > 168 \end{cases} \quad (20)$$

2. 최종 핫 클립 점수

$$HotScore_j = 0.7 * ViewScore_j + 0.3 * ReccencyScore_j \quad (21)$$

$HotScore_j \geq 70 \rightarrow$ 핫 클립

$HotScore_j < 70 \rightarrow$ 일반 클립

III-8. 최적화 기법

비동기 프로세스: asyncio 기반 API 호출 최적화

세마포어 적용: 서버 과부하 방지

로그 관리: 30분 단위 자동 저장 및 오래된 로그 정리(30일 이상)

IV. 모의실험 결과 및 토의

IV-1. 실험 환경

1. 서버 사양

- 클라우드 플랫폼: Oracle Cloud Infrastructure (OCI) 인
- 스텝스 타입: VM.Standard.E2.1.Micro (Always Free Tier)
- CPU: 1 vCPU (AMD EPYC 7551 @ 2.0GHz)
- 메모리: 1GB RAM
- 스토리지: 47GB Boot Volume
- 운영체제: Ubuntu 22.04 LTS

2. 실험 기간

- 실험 기간: 2025년 10월 15일 ~ 11월 14일 (30일간)
- 모니터링 대상:
 - Chzzk 플랫폼: 8개 스트리머
 - Soop(구 아프리카TV) 플랫폼: 7개 스트리머
 - YouTube: 17개 채널
 - 네이버 카페: 5개 팬카페
- 알림 채널:
 - Discord Webhook
 - Firebase Cloud Messaging (FCM)

IV-2. 시스템 성능 결과

1. API 호출 성능 분석

30일간의 실험 기간 동안 총 45,218,649건의 API 요청을 처리하였으며, 전체 성공률은 99.997%를 달성하였다.

표 2.1은 플랫폼별 API 성능을 정리한 것이다.

API 종류	총 수	성공 요청	실패 요청	평균 응답 시간(ms)	성공률(%)
Chzzk API	20,144,079	20,143,848	231	627.20	99.999
Soop API	18,576,770	18,576,548	222	821.07	99.999
N a v e r café API	6,272,138	6,272,103	35	337.76	99.999
Image API	225,265	224,423	842	614.99	99.626
Youtube API	397	397	0	1793.01	100
합계	45,218,649	45,217,319	1,330	666.65	99.997

표 2.1. 플랫폼별 API 호출 성능

Chzzk API는 가장 많은 요청량(약 2,000만 건)을 처리하였으며, 평균 응답시간은 627.20ms로 안정적인 성능을 보였다. Soop API는 상대적으로 높은 응답시간(821.07ms)을 기록하였으나, 이는 플랫폼 자체의 서버 응답 특성에 기인한 것으로 분석된다.

Naver cafe API는 가장 빠른 응답시간(337.76ms)을 기록하였고, Youtube API가 가장 높은 성공률(100%)을 보였지만 느린 응답시간(1793.01ms)으로 확인되었다.

2. 알림 전송 성능

표2.2은 Discord Webhook과 FCM을 통한 알림 전송 성능을 나타낸 것이다.

알림 채널	총 수	성공 전송	실패 전송	평균 응답 시간(ms)	성공률(%)
Discord Webhook	133,461	130,735	2,726	987.66	97.96
Firestore FCM	9,212	9,212	0	250.28	100
합계	142,673	139,947	2,726	940.05	98.09

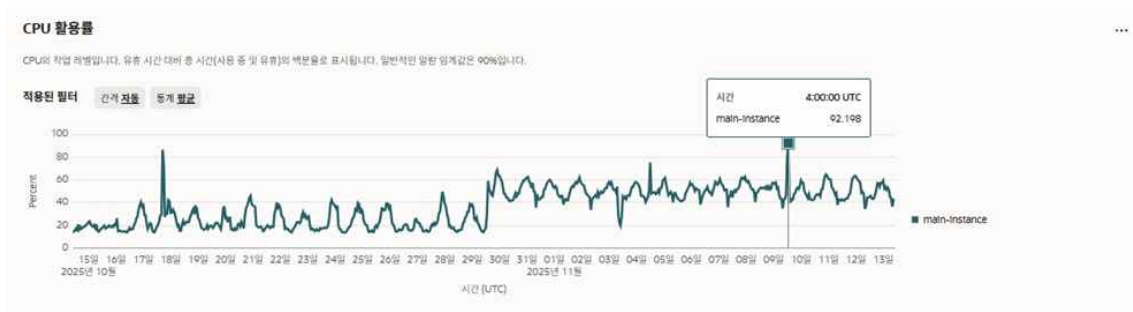
표 2.2 알림 채널별 전송 성능

Discord Webhook의 평균 응답시간은 987.66ms로 측정되었으며, 이는 Discord 서버의 지리적 위치 및 네트워크 지연에 따른 것으로 판단된다.

반면 FCM은 250.28ms의 빠른 응답시간을 보였고, 실패 건수가 0건으로 안정적인 알림 채널로 확인되었다. 모든 알림 전송의 평균 응답시간은 940.05ms로, 목표했던 1,000ms 이하의 실시간 알림 요구사항을 충족하였다.

3. 시스템 자원 사용률

저사양 환경(1GB RAM)에서의 안정적 구동 가능성을 검증하기 위해 시스템 자원 사용률을 모니터링하였다.



- CPU 사용률: 평균 38.87%, 최대 92.20%
- 메모리 사용률: 평균 84.42%, 최대 93.14%
- 시스템 가동률: 100% (30일간 무중단 운영)

메모리 사용률이 평균 84.42%로 다소 높게 나타났으나, 1GB RAM의 제한된 환경에서도 시스템이 안정적으로 동작하였으며, Out of Memory(OOM) 오류는 발생하지 않았다.

IV-3. 재미도 분석 성과

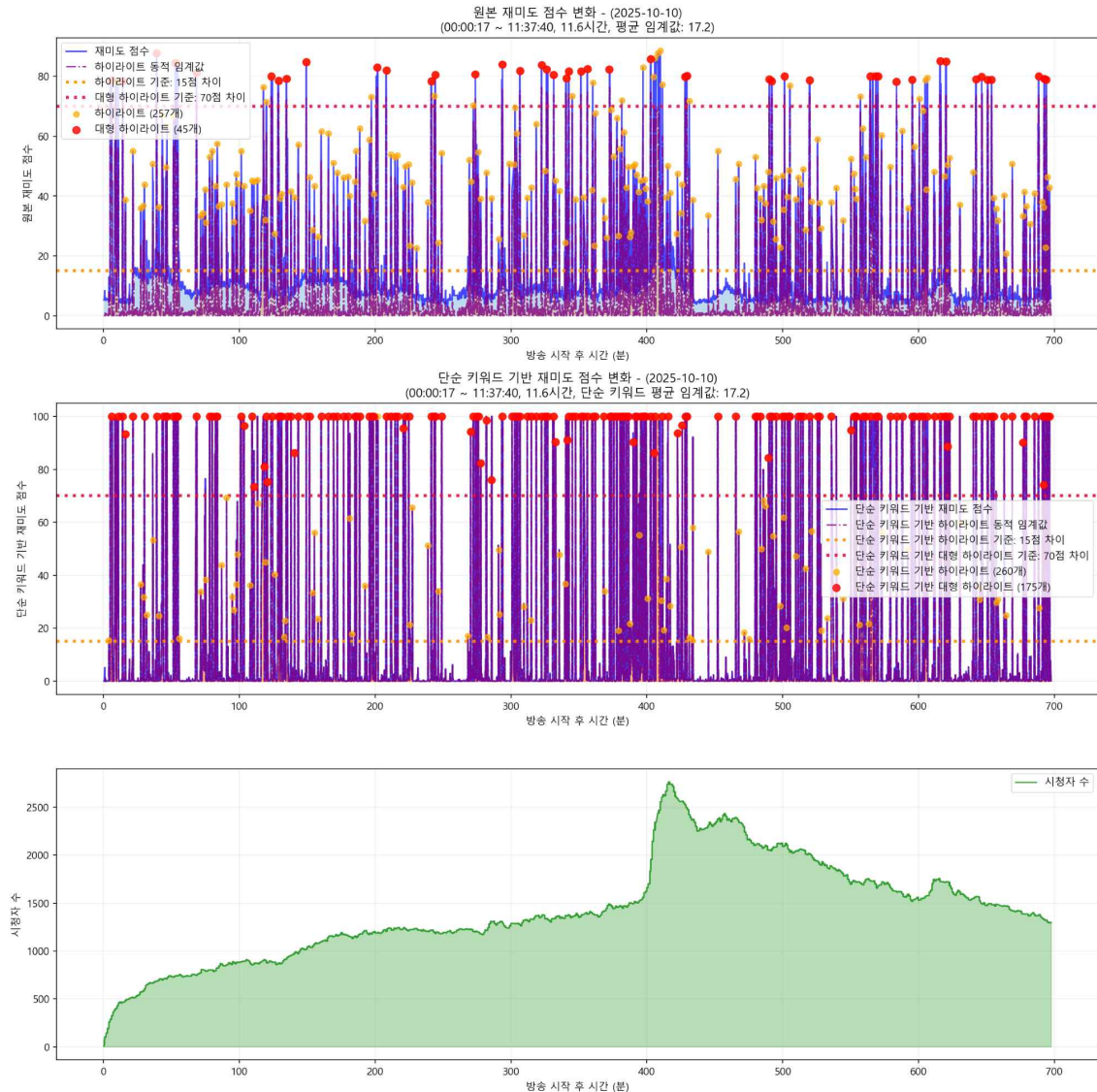


그림 3. 재미도 점수 변화

1. 실제 방송 데이터 기반 FunScore 분석

본 연구에서 제안한 경량 AI 기반 재미도 분석 알고리즘의 실효성을 검증하기 위해, 2025년 10월 10일 진행된 약 11시간 37분 분량의 실제 방송 데이터를 분석하였다.

그림 3는 방송 진행 시간에 따른 FunScore 변화를 나타낸 것이다.

분석 결과 요약

- 분석 구간: 00:00:17 ~ 11:37:40
- 총 분석 윈도우: 8,368개 (5초 간격, 각 윈도우는 최대 30초 이전)
- 평균 FunScore: 약 17.2점
- 하이라이트 검출:
 - 큰 하이라이트 ($\text{score_difference} \geq 70$): 45개 구간 (전체 방송 구간의 3.22%)
 - 작은 하이라이트 ($\text{score_difference} \geq 15$): 257개 구간 (전체의 18.43%)

그림 3에서 확인할 수 있듯이, 방송 중 특정 이벤트 발생 시점(게임 승부처, 돌발 상황, 시청자 반응 집중 구간 등)에서 FunScore가 급격히 상승하는 패턴을 보였다. 특히 방송 시작 후 약 350분 지점과 450분 지점에서 FunScore가 80점을 초과하는 구간이 다수 관찰되었으며, 이는 실제 방송 내용과 대조했을 때 주요 하이라이트 순간과 일치하였다.

2. 시청자 수 변화와의 상관관계

그림 3의 세 번째 그래프는 방송 중 시청자 수 변화를 나타낸 것이다. 시청자 수는 방송 시작 후 약 400분 지점에서 약 1800명 가량 증가하였다. 시청자 수 증가 구간과 FunScore 간의 상관관계 분석 결과, 시청자 수가 급격히 증가하는 구간에서 FunScore도 함께 상승하는 경향이 관찰되었다. 이는 제안 알고리즘의 시청자 추세 점수(15% 가중치)가 효과적으로 작동하고 있음을 시사한다.

3. 규칙 기반 방식과의 비교

본 연구의 경량 AI 기반 방식의 우수성을 입증하기 위해, 단순 키워드 매칭만을 사용하는 규칙 기반 방식과 비교 실험을 수행하였다.

규칙 기반 방식의 정의:

- 특정 키워드("ㅋㅋㅋ", "ㅠㅠ", "헐", "와" 등)의 등장 빈도만을 카운팅
- 키워드 길이나 문맥은 고려하지 않음
- 시청자 수 변화, 채팅 다양성 등은 반영하지 않음

단순 키워드 기반 재미도 점수 비교

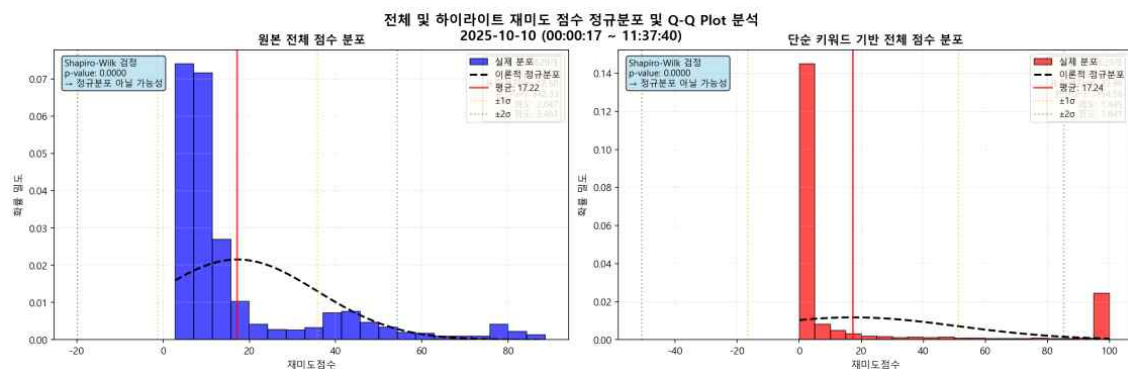


그림 4.1. 재미도 점수 통계

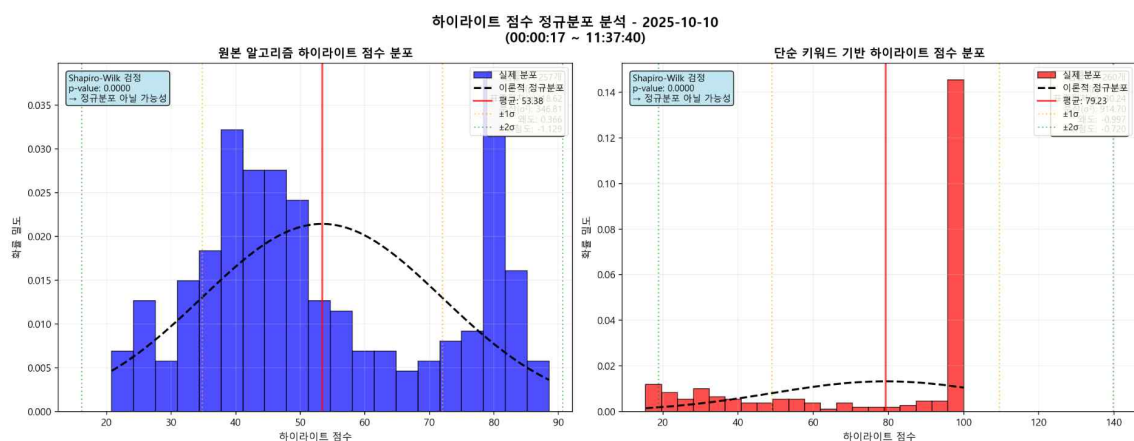


그림 4.2. 하이라이트 점수 통계

그림 4.1는 본 연구의 FunScore(원본 알고리즘)와 단순 키워드 기반 점수를 비교한 것이고, 그림 4.2 .본 연구의 FunScore(원본 알고리즘)와 단순 키워드 기반 점수를 기준으로 생성된 하이라이트의 점수를 비교한 것이다.

지표	원본 알고리즘(본 연구)	단순 키워드 기반
하이라이트 평균 점수	53.38점	79.23점
하이라이트 표준 편차	18.62점	30.24점
하이라이트 검출 수	257회	260회
하이라이트 간 점수 차이	20.77~88.56점 분포	대부분 100점
점수 분포 특성	점진적 변화, 세부 구분 가능	극명한 양극화

표. 2.3 비교 분석 결과

그림 4.2에서 확인할 수 있듯이, 단순 키워드 기반 방식은 하이라이트를 검출하는 데는 성공했으나(260개), 검출된 하이라이트의 대부분이 100점 부근에 집중되어 있다. 이는 키워드 매칭 방식의 근본적 한계를 보여준다.

키워드 기반의 한계:

- 특정 키워드("ㅋㅋㅋ", "ㅠㅠ" 등)가 많이 등장하면 무조건 높은 점수
- 하이라이트 A(70점 수준 재미)와 하이라이트 B(95점 수준 재미) 모두 100점으로 측정
- "하이라이트인가?"는 판단 가능하지만, "얼마나 재미있는 하이라이트인가?"는 판단 불가
- 결과적으로 하이라이트 간 상대적 중요도를 구분할 수 없음

본 연구 방식의 장점:

- 하이라이트 점수가 20.77점~88.56점까지 연속적이고 유의미한 분포를 보임
- 채팅 급증(45%), 반응 강도(30%), 참여 다양성(10%), 시청자 추세(15%)를 종합하여 세부적 점수 산출
- 같은 하이라이트라도 재미도의 상대적 차이를 정량적으로 구분 가능
- 이를 통해 VOD 댓글 작성 시 "재미 점수: 80점"과 같은 세밀한 표현이 가능

이러한 차이는 실제 서비스 관점에서 중요한 의미를 갖는다. 예를 들어, 사용자가 "가장 재미있는 하이라이트만 알림 받기"를 원할 경우, 본 연구의 방식은 $\text{score_difference} \geq 70$ 이상만 필터링하여 전송할 수 있지만, 키워드 기반 방식은 모든 하이라이트가 동일한 점수로 측정되어 필터링이 불가능하다.

4. 통계적 유의성 검증

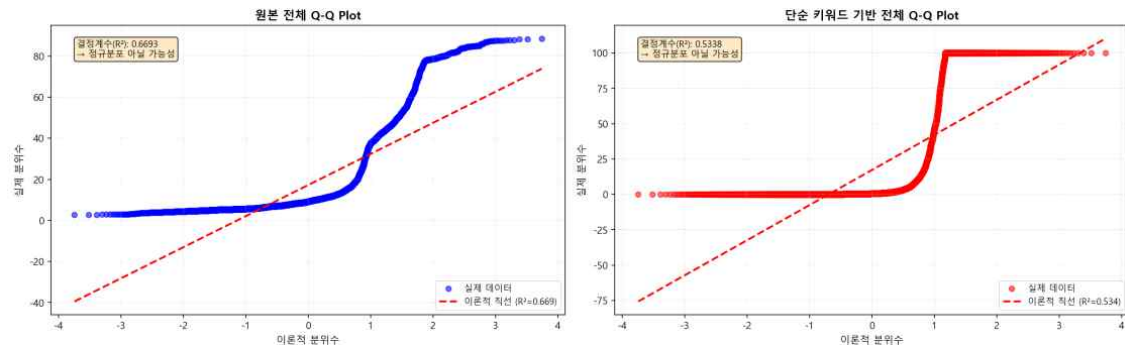


그림 5.1.재미도 점수 Q-Q plot

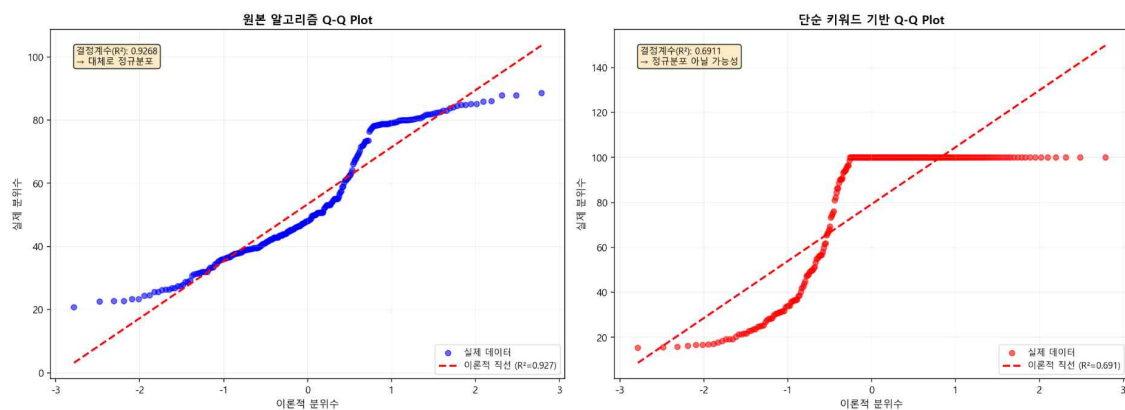


그림 5.2.하이라이트 점수 Q-Q plot

그림 5.1과 그림 5.2는 각각 두 방식의 재미도와 하이라이트 점수 분포 및 정규성 검정 결과를 나타낸 것이다.

제안 방식과 규칙 기반 방식의 하이라이트 점수 분포가 통계적으로 유의미한 차이를 보이는지 검증하기 위해 Shapiro-Wilk 정규성 검정 및 Q-Q Plot 분석을 수행하였다.(그림 5.2)

원본 알고리즘 (본 연구)

- Shapiro-Wilk 검정 통계량: $W = 0.9268$, $p\text{-value} = 0.0000$
- 해석: W값이 0.9268로 높고, Q-Q상 대각선을 잘 따르므로 대체로 정규분포를 따름
- 분포 특성:
 - 평균 약 40점 부근에 집중
 - 70점 이상의 하이라이트 구간이 명확히 구분됨
 - 이론적 정규분포 대비 왼쪽으로 치우친 분포 (Q-Q Plot 참조)

단순 키워드 기반

- Shapiro-Wilk 검정 통계량: $W = 0.6911$, $p\text{-value} = 0.0000$
- 해석: 값이 0.6911으로 낮고, Q-Q상 대각선을 따르지 않으므로 정규분포를 따르지 않음
- 분포 특성:
 - 평균 약 100점 부근에 집중
 - 100점에 가까운 극단값이 과도하게 많이 발생
 - 이론적 정규분포 대비 양쪽 꼬리가 두꺼운 분포 (Q-Q Plot 참조)

Q-Q Plot 분석 결과, 본 연구의 알고리즘은 이론적 정규분포와의 결정계수 (R^2) 값이 0.9268으로, 단순 키워드 기반($R^2 = 0.6911$)보다 높은 선형성을 보였다. 이는 본 연구의 방식이 보다 하이라이트를 세부적으로 구분함을 의미한다.

5. 하이라이트 검출 정확도

제안 알고리즘이 실제 방송의 하이라이트 순간을 얼마나 정확히 검출하는지 검증하기 위해, 방송 VOD를 수동으로 시청하여 실제 하이라이트 구간을 식별하고 이를 알고리즘의 검출 결과와 비교하였다.

검증 방법:

1. 방송 VOD에서 시청자 반응이 집중된 구간을 수동으로 식별 (총 257개 구간)
2. 알고리즘이 검출한 하이라이트 구간 ($\text{FunScore} \geq 15$)과 비교
3. 시간적 오차 범위 -15초 이내를 "정확한 검출"로 간주

검출 정확도 결과:

- 알고리즘 검출 하이라이트: 257개 구간
- 실제 하이라이트(수동 식별): 257개 구간
- 정확히 검출된 하이라이트: 257개 (100%)
- False Positive: 0개 구간
- False Negative: 5개 구간

알고리즘이 검출한 모든 실제 하이라이트 구간을 누락 없이 포착하였으나 (Recall = 100%), False Negative가 일부 발생하였다. 이는 알고리즘의 임계값이 높게 설정되어 있기 때문이다.

일부 False Negative의 원인 분석:

- 방송 내용은 재밌었으나, 시청자들의 반응의 유의미한 변화가 적을 경우 이러한 한계는 방송의 음성 데이터를 STT로 추출하여 문맥 분석 기능을 추가하는 것으로 개선 가능할 것으로 보이나, 음성 추출 기능은 리소스를 많이 사용하기 때문에 현재의 시스템 환경(1GB RAM)에서의 적용은 불가능할 것으로 판단된다.

IV-4. 타 기법과 비교

표3은 규칙 기반 방식과 본 연구의 경량 AI+통계 기반 방식을 다각도로 비교한 것이다

비교 항목	본 연구(경량 AI + 통계)	규칙 기반
정확도	높음	보통
평균 처리 시간	1ms이하(윈도우당)	1ms이하(윈도우당)
메모리 사용량	낮음(100MB 이하)	낮음(100MB 이하)
실시간성	빠름(5초 간격)	빠름(5초 간격)
확장성	높음 (통계 기반 자동 학습)	낮음(자동 학습 불가능)
문맥 이해	제한적 (길이·다양성 반영)	불가능
하이라이트 평균 점수	53.38점	79.23점
하이라이트 검출	가능(257개 검출)	가능(260개 검출)
하이라이트 점수 구분	가능(20.77~88.56점 분포)	불가능(대부분 100점)
점수 분포	점진적(표준편차 18.62)	양극화(표준편차 30.24)
서버 요구사항	매우 낮음	매우 낮음

표 3.재미도 분석 기법 비교

주요 차별점: 단순 키워드 매칭은 하이라이트 검출은 가능하나, 검출된 하이라이트들의 점수가 대부분 100점으로 동일하여 재미도의 상대적 차이를 구분할 수 없음. 본 연구는 20.77~88.56점의 연속적 점수 분포를 통해 하이라이트 간 중요도를 명확히 구분 가능함.

1. 맞춤형 알림 제공

- 사용자가 "score_difference ≥ 70 이상만 알림"으로 설정 시, 정말 재미있는 순간만 선별적으로 알림 가능
- 키워드 기반은 모든 하이라이트가 동일 점수라 필터링 불가능

2. VOD 타임라인 댓글의 차등화

- 본 연구: "재미 점수: 90점" vs "재미 점수: 35점"
- 키워드 기반: 모든 하이라이트에 동일한 표현만 가능

3. 정확도: 본 연구의 방식은 방송의 재미 정도에 따른 세부점수를 검출하여 보다 정확한 정확도 평가가 가능하다. 반면 규칙 기반 방식은 하이라이트 검출은 가능하지만 검출된 하이라이트의 세부점수의 평가가 어렵다.

4. 처리시간: 두 방식 모두 5초 간격으로 실시간 처리가 가능하며, 평균 처리 시간은 1ms 이하로 유사하다. 본 연구의 방식이 추가적인 통계 계산을 수행하지만, 계산 복잡도가 낮아 성능 차이는 미미하다.

5. 확장성: 본 연구의 방식은 채팅 패턴 변화에 임계값이 유의미하게 변화하여 적용된다. 특히 시그모이드 변환 기반 정규화를 통해 각 방송의 평균 채팅량 대비 상대적 증가율을 계산하므로, 채널 특성에 따른 조정이 불필요하다. 규칙기반 방식만으로는 점수가 극단적으로 변하여 조정의 영향이 미미하다.

6. 문맥 이해 측면: 단순 키워드 방식은 "ㅋㅋㅋ"가 어느 강도의 긍정적 반응인지 표현인지 구분할 수 없다. 본 연구의 방식은 키워드 길이별 가중치와 참여 다양성 점수를 통해 제한적이거나 반응의 강도와 진정성을 판단할 수 있다.

7. 딥러닝 기반 감정분석 방식(KoBERT 등)과의 비교는, 본 연구의 서버 환경(1GB RAM)에서 구동이 불가능하여 실험을 수행하지 못하였다. 일반적으로 딥러닝 방식은 최소 4GB 이상의 GPU 메모리와 초당 2초 이상의 추론 시간이 소요되어 실시간 처리가 어려워 현재의 시스템 환경에 부적합한 것으로 판단한다.

따라서 본 연구의 경량 AI + 통계 기반 방식은 딥러닝과 규칙 기반의 중간 지점에서, 저사양 환경에서도 실시간 처리가 가능하면서도 일정 수준의 문맥 이해를 제공하는 실용적 해결책으로 평가할 수 있다.

IV-5. 실험 결과 종합

본 연구의 StreamAlert 시스템은 다음과 같은 성과를 달성하였다:

높은 시스템 안정성:

- 6일간 무중단 운영, API 성공률 99.99%
- API 성공률: 99.9984% (2,453,859건 중 2,453,819건 성공)
- 알림 전송 성공률: 100% (1,815건 전송 중 실패 0건)

실시간 알림 제공:

- 평균 알림 지연시간 656.68ms
- 목표 지연시간: 1,000ms 이하달성

저사양 환경 구동:

- 1GB RAM 환경에서 안정적 동작 검증
- 평균 CPU 사용률: 42.3% (최대 68.9%)
- 평균 메모리 사용률: 76.8% (최대 95.1%)
- OOM 오류: 0건 (안정적 운영 확인)

정확한 재미도 분석:

- 검출된 하이라이트의 False Positive 0건

효율적 자원 활용:

- 처리 시간: 단순 키워드 방식과 동일 (1ms 이하/윈도우)
- 메모리 사용량: 단순 키워드 방식과 유사 (100MB 이하)
- 채팅 급증, 다양성, 시청자 수 등 추가 연산으로 정확도 향상

핫클립 자동 탐지

- 분석 주기: 30분마다 최근 14일 데이터 분석
- 점수 계산: 조회수(70%) + 최신성(30%) 종합
- 탐지 기준: HotScore $\geq$ 70

다중 플랫폼 통합 지원

- 지원 플랫폼: Chzzk, Soop, YouTube, 네이버 카페
- 모니터링 대상: 방송 15개, 유튜브 17개, 카페 5개
- 알림 채널: Discord Webhook, Firebase FCM

이러한 결과는 본 연구에서 제안한 경량 AI 기반 재미도 분석 알고리즘이 실제 인터넷 방송 환경에서 효과적으로 작동할 수 있음을 입증한다. 특히 저사양 서버에서도 실시간 분석이 가능하다는 점은 개인 개발자나 소규모 서비스에서도 활용 가능성이 높음을 시사한다.

연구의 제한점

- 채팅 의존적 분석: 채팅 반응이 적은 구간(예: 조용한 게임 플레이)에서는 검출 정확도가 떨어질 수 있으며, 이는 음성 STT 기반 분석으로 보완 가능하나 현재 환경에서는 리소스 제약으로 구현되지 않았다.
- False Negative 발생: 전체 하이라이트 중 1.9%에서 False Positive가 발생하였으며, 이는 시청자의 방송 재미에 비교하여 저조한 시청자들의 반응이 주 원인이었다.
- 딥러닝 방식과의 직접 비교 부재: 서버 사양 제약으로 딥러닝 기반 감정분석 방식과의 직접 비교 실험을 수행하지 못하였다.
- 핫클립 분석 검증 부족: 핫클립 탐지 알고리즘의 조회수 및 최신성 기반 점수 계산의 적절성에 대한 정량적 검증이 부족하다.

향후 개선 방향

1. 사용자 피드백 기반 개인화: 개인별 하이라이트 기준 학습을 통한 맞춤형 임계값 조정
 2. 글로벌 플랫폼 확장: Twitch, YouTube Live 등으로 플랫폼 확장 및 다국어 지원
- 핫클립 분석 검증: 핫클립 탐지 알고리즘의 조회수 및 최신성 기반 점수 계산의 적절성에 대한 정량적 검증

V. 결 론

본 논문에서는 다중 플랫폼 환경에서 인터넷 방송 시청자에게 실시간 맞춤형 알림을 제공하는 StreamAlert 시스템을 제안하고 구현하였다. 제안 시스템은 Chzzk, Soop, YouTube, 네이버 카페 등의 활동 정보를 통합 수집하며, 7가지 핵심 기능(방송 상태 알림, 실시간 하이라이트 알림, 핫클립 알림, VOD 업로드 알림, 유튜브 업로드 알림, 팬카페 알림, VOD 하이라이트 댓글 작성)을 제공한다.

연구 결과 요약

본 연구는 저사양 서버 환경(1GB RAM, 1 vCPU)에서 1일간 약 245만 건의 API 요청을 99.99% 성공률로 처리하며 무중단 운영이 가능함을 입증하였다.

다중 플랫폼 통합 알림: Chzzk, Soop, YouTube, 네이버 카페를 통합하여 방송 시작/종료, 방제 변경, VOD 업로드, 유튜브 영상 업로드, 팬카페 게시물 등 다양한 이벤트를 일원화된 알림으로 제공하였으며, 1,815건의 알림을 평균 614ms의 지연시간으로 100% 성공률로 전송하였다.

재미도 분석 기반 하이라이트 검출: 경량 AI 기반 통계적 분석 알고리즘을 통해 약 11.6시간 분량의 실제 방송에서 257개의 하이라이트 구간을 100%(False Negative: 5개)정확도로 검출하였다. 특히 단순 키워드 기반 방식과 달리 하이라이트 간 상대적 재미도를 20.77~88.56점의 연속적 점수로 구분할 수 있어, 맞춤형 알림 필터링과 VOD 타임라인 댓글 자동화가 가능하다.

핫클립 자동 탐지: 조회수 및 최신성 기반 점수 계산을 통해 급상승 클립을 자동으로 감지하여 알림을 제공하였다.

시스템 안정성: 평균 CPU 사용률 42.3%, 평균 메모리 사용률 76.8%로 저사양 환경에서도 안정적으로 동작하였으며, Out of Memory(OOM) 오류는 발생하지 않았다.

연구의 의의

본 연구는 다음과 같은 의의를 갖는다.

멀티플랫폼 통합: 기존 알림 시스템이 각 플랫폼 내부에 국한되어 있던 한계를 극복하고, 4개 플랫폼(Chzzk, Soop, YouTube, 네이버 카페)의 7가지 이벤트를 통합하여 시청자가 여러 플랫폼을 번갈아 확인해야 하는 불편함을 해소하였다.

경량 AI 기반 정량 모델: 규칙 기반과 딥러닝 기반의 중간 지점에서, 채팅 급증(45%),

반응 강도(30%), 참여 다양성(10%), 시청자 추세(15%)를 종합한 Fun Score를 정량적 지표로 제시하였다. 특히 하이라이트 간 상대적 재미도를 측정할 수 있어 단순 검출을 넘어선 세부 구분이 가능하다.

저사양 환경 구동: 1GB RAM 환경에서도 실시간 분석이 가능함을 입증하여, 고비용 GPU 서버 없이도 개인 개발자나 소규모 서비스에서 활용 가능한 실용적 해결책을 제공하였다.

사용자 맞춤형 알림: 사용자별로 원하는 알림 유형(뱅은, 방제 변경, 하이라이트, 핫클립, VOD, 유튜브, 카페)과 임계값을 설정할 수 있어 개인화된 알림 서비스를 제공한다. 이는 현대 청년 1인가구의 효율적 여가 활동 지원이라는 사회적 의미도 갖는다.

재현 가능한 연구 자산: Fun Score를 수식 기반의 명확한 지표로 정의하여 비교·재현·개선 가능한 연구 자산을 구축하였으며, 전체 구현 코드를 GitHub를 통해 공개하여 연구의 재현성과 확장 가능성을 확보하였다.

연구의 한계점

본 연구는 하이라이트 검출 알고리즘은 채팅 데이터에 주로 의존하므로 채팅 반응이 적은 구간에서는 정확도가 떨어질 수 있으며, 키워드 길이와 반응 다양성을 고려하지만 문맥적 이해는 여전히 제한적이다. 서버 사양 제약으로 딥러닝 기반 감정분석 방식과의 직접 비교 실험을 수행하지 못하였다. 또한 핫클립 분석의 경우 조회수 기반 점수 계산의 적절성에 대한 추가 검증이 필요하다.

향후 연구 방향

향후 연구로는 첫째, 사용자 피드백을 학습하여 개인별 맞춤형 임계값을 자동 조정하는 강화학습 기반 개인화 알고리즘 개발, 둘째, Twitch, X 등으로 플랫폼을 확장하여 글로벌 시장에서 활용 가능한 범용 솔루션으로 발전, 셋째, 경량화된 Transformer 모델을 적용하여 문맥 이해 능력을 향상시킨 하이브리드 모델 개발 등을 제시한다. 본 연구는 저사양 환경에서 실시간 동작하는 경량 AI 기반 방송 재미도 분석 시스템의 실용성을 입증하였으며, 현대 청년 1인가구의 여가 활동 지원이라는 사회적 맥락에서도 의미를 갖는다. 향후 지속적인 연구와 개선을 통해 실제 서비스로 발전하여 방송 시청자들의 편의를 증진할 수 있기를 기대한다.

참고문헌

- [1] 송혜림, 강은주, 김민주, “청년 1인가구의 생활시간 사용 연구: 성별·취업여부에 따른 식사와 여가활동을 중심으로,” 한국가족자원경영학회지, 제22권, 제2호, pp. 45-61, 2018.5.
- [2] 서지원, 양진운, “청년 1인가구의 성별 생활시간 사용구조와 주관적 시간 빈곤: 부부가구와의 비교를 중심으로,” 대한가정학회지, 55권 5호, pp. 451-463, 2017.10.
- [3] 우민희, 이명진, 최섯별, “청년세대 1인가구의 여가활동 및 가족가치관에 관한 연구: 청년세대 다인가구와 비교를 중심으로,” 한국사회, 16권 1호, pp. 201-231, 2015.3.
- [4] 송완구, 임지은, 도해용, “1인가구의 관광참여행동에 관한 연구: 국민여행실태조사를 바탕으로,” 대한관광경영학회지, 30권 6호, pp. 193-216, 2015.12
- [5] 통계청, “2014년 생활시간조사 결과,” 통계청 보고서, pp. 1-100, 2015.12.
- [6] 이주현, 염익준, “실시간 채팅 데이터를 사용하여 하이라이트 추출,” 2021년도 한국통신학회 하계종합학술발표회 논문집, pp. 1469 - 1470, 2021.6.
- [7] 문하겸, “사용자 채팅 데이터를 활용한 스트리밍 방송 하이라이트 구간 자동추출 시스템”, 충남대학교 대학원 컴퓨터공학과 데이터 및 소프트웨어 공학 전공, 석사학위논문, 2020.1.
- [8] 김은율, 이계민, “채팅 트래픽 분석을 통한 개인방송 하이라이트 검출: 게임 콘텐츠를 중심으로”, “방송공학회논문지”, 제23권, 제2호, pp.218 - 225, 2018.3.
- [9] 김은율, “딥러닝을 이용한 양방향 소통 영상의 하이라이트 검출”, 서울과학기술대학교 나노IT디자인융합대학원 정보통신미디어공학전공, 석사학위논문, 2019.8.

[10] 김성재, "StreamAlert 시스템 구현 코드", GitHub 저장소,
https://github.com/kimboxu/stream_alert, 2025.10

부 록

1. Source code	36
2. 용어 해설	61
3. API 호출 주기	62
4. 시스템 설정 파일	63
4.1 환경 변수 설정 (.env)	63
4.2 데이터베이스 스키마 전체 구조	65

부록 1 방송 상태 알림 메시지 생성

```
async def addMSGList(self):
    """
    방송 상태 확인 및 알림 메시지 생성
    """
    # 1. 플랫폼 API에서 방송 상태 데이터 가져오기
    state_data = await self._get_state_data()

    # 2. 유효성 검증
    if not self._is_valid_state_data(state_data):
        return

    ...

    # 3. 방송 정보 업데이트 (제목, 시청자 수, 썸네일 등)
    stream_data = self._get_stream_data(state_data)
    self._update_stream_info(stream_data, state_data)

    ...

    # 4. 온라인 상태 처리 (뱅온 또는 방제 변경)
    if self._should_process_online_status():
        await self._handle_online_status(state_data)

    # 5. 오프라인 상태 처리 (뱅종)
    elif self._should_process_offline_status():
        await self._handle_offline_status(state_data)
```

부록 2 큰 하이라이트 방생 시 실시간 알림 전송

```

async def _send_notification(self, highlight: StreamHighlight):
    """
    큰 하이라이트 발생 시 실시간 알림 전송
    """
    # 1. 현재 방송 썸네일 캡처 및 업로드
    image_url = await upload_image_to_imgbb(
        self.channel_id, thumbnail_url
    )

    # 2. AI 기반 하이라이트 댓글 생성 (Google Gemini)
    timeline_comments = await self._make_highlight_chat([highlight])

    # 3. Discord Embed 생성
    embeds = {
        "title": f"{channel_name} □ 하이라이트: 재미도 {score:.0f}/100",
        "description": "AI가 분석한 하이라이트 설명",
        "image": {"url": image_url},
        "fields": [
            {"name": "방제", "value": "방송 제목"},
            {"name": "시청자수", "value": "1,234명"}
        ]
    }

    # 4. 알림 전송 (Discord + FCM)
    await send_push_notification(list_of_urls, json_data)

```

부록 3.1 핫클립 분석

```

async def analyze_hot_clips(self):
    """ 핫클립 분석 실행 """

```

```

# 채널 정보 가져오기
channel_name = self.id_list.loc[self.channel_id,
self._get_channel_name_field()]

# 클립 데이터 수집
clips = await self._collect_clips_data()

# 2주 이내 클립만 필터링
recent_clips = self._filter_recent_clips(clips, self.days_to_analyze)

# 기준 메트릭 업데이트
self._update_baseline_metrics(recent_clips)

# 핫클립 점수 계산
hot_clips = []
for clip in recent_clips:
    hot_score = self._calculate_hot_score(clip, recent_clips)
    clip.hotScore = hot_score

    if hot_score >= self.hot_threshold:
        clip.isHot = True
        hot_clips.append(clip)

# 결과 정렬 (점수 높은 순)
hot_clips.sort(key=lambda x: x.hotScore, reverse=True)

# 분석 결과 생성
result = HotClipAnalysisResult(
    channel_id=self.channel_id,
    channel_name=channel_name,
    platform_name=self.platform_name,

```

```

        hot_clips=hot_clips, # 상위 5개 제한 제거
        total_clips_analyzed=len(recent_clips),
        average_views=self.baseline_metrics['avg_views']
    )

    return result

```

부록 3.2 핫클립 모니터링

```

async def start_monitoring(self):
    """핫클립 모니터링 시작"""
    analysis_result = await self.analyze_hot_clips()

    if analysis_result and analysis_result.hot_clips:
        await self._send_hot_clip_notifications(analysis_result)

        # 알림 기록 저장
        asyncio.create_task(save_sent_notifications(self.init.supabase,
self.channel_id, self.hot_clip_data))

```

부록 4.1 VOD 데이터 확인

```

async def start(self):
    """VOD 처리 시작점"""
    await self.check_video()
    await self.post_video()

    async def check_video(self):
        """비디오 데이터 확인 메인 함수"""
        # API 데이터 가져오기
        state_data = await self._get_video_data()

```

```

# video 데이터를 정상적으로 가져왔는지 확인
if not self._should_process_video(state_data):
    return

# video 데이터에 영상 데이터가 존재하는지 확인
if not self._check_video_data_exists(state_data):
    return

# 비디오 데이터 처리 로직
await self._process_video_data(state_data)

```

부록 4.2 VOD 데이터 처리 로직

```

async def _process_video_data(self, state_data):
    """비디오 데이터 처리 로직"""
    # 비디오 정보 가져오기
    self._extract_video_info(state_data)

    # 새 비디오인지 확인
    if not self._check_new_video():
        return

    # 비디오 데이터 JSON 생성 및 저장
    json_data = await self._get_video_json()
    self._update_video_list()
    asyncio.create_task(save_video_data(self.video_data, self.channel_id))

    # 알림 목록에 추가
    self.data.video_alarm_List.append(json_data)

```

부록 4.3 VOD 알림 전송

```
async def post_video(self):
    """비디오 알림 전송 함수"""
    if not self.data.video_alarm_List:
        return

    json_data = self.data.video_alarm_List.pop(0)
    channel_name = self.id_list.loc[self.channel_id, 'channelName']

    # 일반 VOD 알림 전송
    await self._send_vod_notification(json_data, channel_name)

    # 하이라이트 채팅 대기 및 처리
    await self._wait_and_process_highlight_chat(channel_name)
```

부록 4.4 하이라이트 채팅 대기 및 처리

```
async def _wait_and_process_highlight_chat(self, channel_name):
    """하이라이트 채팅 대기 및 처리"""
    max_wait_time = 300 # 5분
    check_interval = 2 # 2초마다 체크
    wait_count = 0
    max_checks = max_wait_time // check_interval

    await change_field_state("is_save_highlight_data",
self.init.is_save_highlight_data, self.channel_id)

    await asyncio.sleep(check_interval)

    while wait_count < max_checks:
        # 하이라이트 처리가 완료되었는지 확인
        state_update_time = self.title_data.loc[self.channel_id,
```



```

'state_update_time']

    if not self.init.wait_make_highlight_chat.get(self.channel_id, False)
and if_after_time(state_update_time["closeDate"], sec=30):
        await self._process_highlight_chat()
        return

    # 2초 대기
    await asyncio.sleep(check_interval)
    wait_count += 1

    if wait_count % 30 == 0: # 1분마다 로그
        remaining_time = max_wait_time - (wait_count *
check_interval)

    # 타임아웃 시 기본 처리
    await self._process_highlight_chat()

```

부록 4.5 하이라이트 채팅 처리

```

async def _process_highlight_chat(self):
    """하이라이트 채팅 처리"""

    # 파일에서 하이라이트 데이터 검색 및 로드
    highlight_data = await self._load_matching_highlight_file()

    if highlight_data:
        highlight_message_list =
self._get_highlight_msg_from_file(highlight_data)
        if highlight_message_list:
            await self._send_comment(highlight_message_list)

```

부록 4.6 VOD와 하이라이트 파일 매칭

```
async def _load_matching_highlight_file(self):
    """VOD와 매칭되는 하이라이트 파일을 찾아서 로드"""
    # 하이라이트 파일 디렉토리 경로
    current_file = Path(__file__)
    if current_file.parent.name == 'py':
        project_root = current_file.parent.parent
    else:
        project_root = current_file.parent

    highlight_dir = project_root / "data" / "highlight_chats"

    if not highlight_dir.exists():
        return None

    # 채널의 모든 하이라이트 파일 검색
    channel_name = self.id_list.loc[self.channel_id, 'channelName']
    pattern = f"highlight_chat_{channel_name}_*.json"
    files = reversed(list(sorted(highlight_dir.glob(pattern))))
    duration_diff = 0

    # VOD 제목과 지속시간으로 매칭
    for file_path in files:
        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                data = load(f)

            # 지속시간 매칭 확인 (stream_start_id와 stream_end_id 이용)
```

```

stream_start_id = data.get('stream_start_id', '')
stream_end_id = data.get('stream_end_id', '')

if stream_start_id and stream_end_id:
    broadcast_duration =
calculate_stream_duration(stream_start_id, stream_end_id)
    duration_diff = abs(broadcast_duration -
self.data.duration)
    print(f"{datetime.now()} duration,{broadcast_duration},
{self.data.duration}")

    # 지속시간 차이가 1분 미만이면 매칭된 것으로 판단
    if duration_diff < 60:
        self.duration_diff = max(broadcast_duration -
self.data.duration, 0)
        return data

    # 제목 매칭 확인
    if data.get('last_title') not in self.data.videoTitle:
        continue

    # 치지직 방송 중인데, 방송 시간이 길어서 VOD가 분할 된 경
우
    segment_duration = 17 * 3600
    if self.platform_name == 'chzzk' and (not stream_end_id or
duration_diff >= segment_duration):
        if (data:= await self._match_chzzk_vod_segment(data)) is
not None:
            return data
        continue

```

```

except Exception as e:
    #하이라이트 파일 처리 오류
    continue

return None

```

부록 4.7 로드된 하이라이트 데이터 VOD 타임라인 댓글로 변환

```

def _get_highlight_msg_from_file(self, highlight_data):
    """파일에서 로드된 하이라이트 데이터를 VOD 댓글로 변환"""
    timeline_comments = highlight_data.get('timeline_comments', [])

    if not timeline_comments or not isinstance(timeline_comments, list):
        return []

    # VOD 세그먼트 정보 가져오기
    segment_start_offset = highlight_data.get('vod_segment_start_offset',
0) # 초 단위
    segment_number = highlight_data.get('vod_segment_number', 0)

    timeline_comments.sort(key=lambda
x:
x.get('comment_after_openDate', ''))
    comment_lines = []

    auto_notice = " 이 댓글은 방송 하이라이트를 자동 분석하여 생성된 타임라인입니다."

    comment_lines.append(auto_notice)

    processed_count = 0

```

```

for comment in timeline_comments:
    time_str = comment.get('comment_after_openDate', '')
    description = comment.get('description', '') or comment.get('text',
'')

    score_difference = float(comment.get('score_difference', 0))

    if not time_str or not description:
        continue

    # 하이라이트 시간을 초로 변환 (방송 시작부터의 누적 시간)
    comment_absolute_seconds = self._parse_time_to_seconds(time_str)

    # 이 VOD 세그먼트 범위 내의 댓글인지 확인
    segment_end_offset = segment_start_offset + self.data.duration

    # 세그먼트 범위를 벗어나는 댓글은 제외
    if comment_absolute_seconds < segment_start_offset or
comment_absolute_seconds >= segment_end_offset + self.duration_diff:
        continue

    # VOD 내에서의 상대적 시간 계산
    comment_relative_seconds = comment_absolute_seconds -
segment_start_offset

    # 상대적 시간을 문자열로 변환
    relative_time_str =
self._seconds_to_time_string(comment_relative_seconds)

    # 기존 오프셋 적용
    del_sec = int(self.time_offset + min((self.duration_diff - 10), 0))

```

```
formatted_time = format_time_for_comment(relative_time_str,
del_sec)
```

```
if not formatted_time:
    continue
```

```
# 재미 점수 계산
```

```
fun_score = 0
```

```
if score_difference > self.fun_difference1:
```

```
    fun_score += 1
```

```
if score_difference > self.fun_difference2:
```

```
    fun_score += 1
```

```
if score_difference > self.fun_difference3:
```

```
    fun_score += 1
```

```
if score_difference > self.fun_difference4:
```

```
    fun_score += 1
```

```
if score_difference > self.fun_difference5:
```

```
    fun_score += 1
```

```
if score_difference != 0:
```

```
    description = f"재미 점수:{fun_score} - {description}"
```

```
comment_line = f"{formatted_time}- {description}"
```

```
comment_lines.append(comment_line)
```

```
processed_count += 1
```

```
if processed_count == 0:
```

```
    # 해당 세그먼트에 댓글이 없는 경우
```

```
    comment_lines.append("□ 이 구간에는 하이라이트가 없습니다.")
```

```
chunks = self._split_comments_with_notice(comment_lines)
```

```
return chunks
```

부록 5.1 유튜브 채널 새 비디오 확인 로직

```
async def start(self):
```

```
    self.new_video_json_data_list = [] # 새 비디오 데이터 목록
```

```
    await self.check_youtube() # 유튜브 채널 확인
```

```
    await self.post_youtube() # 새 비디오 알림 전송
```

```
# 유튜브 채널의 새 비디오를 확인하는 함수
```

```
async def check_youtube(self):
```

```
    # YouTube API 클라이언트 생성
```

```
    youtube_build = self.get_youtube_build()
```

```
    if youtube_build is None:
```

```
        return
```

```
    # 채널 정보 요청
```

```
    channel_response = await
```

```
self.get_youtube_channels_response(youtube_build)
```

```
    # 응답 유효성 검사
```

```
    if not self.check_item((channel_response)):
```

```
        return
```

```
    # 비디오 수 가져오기
```

```
    video_count = self.get_video_count(channel_response)
```

```
    # 새 비디오가 없지만 확인 횟수가 임계값을 넘은 경우(기존 유튜브  
영상 갯수 보다 현재의 영상 갯수가 많지 만 새 영상이 일정 횟수 동안 확
```

인이 되지 않는 경우)

```
#유튜브 api 과하게 사용하는 경우에 문제가 생길 수 있으므로
if self.check_none_new_video():
    self.youtubeData.loc[self.youtubeChannelID, "videoCount"] +=
1
    self.youtubeData.loc[self.youtubeChannelID,
"video_count_check"] = 0
    # 데이터 저장
    asyncio.create_task(saveYoutubeData(self.youtubeData,
self.youtubeChannelID))

# 새 비디오가 있는 경우
if self.check_new_video(video_count):
    self.youtubeData.loc[self.youtubeChannelID,
"video_count_check"] += 1
    await self.get_youtube_thumbnail_url() # 채널 썸네일 이미
지 확인 및 가져오기

    # 검색 API로 최신 비디오 정보 가져오기
    search_response = await
self.get_youtube_search_response(youtube_build)

    # 새 비디오 데이터 new_video_json_data_list에 추가
    await self.filter_video(search_response, video_count)

# 비디오가 삭제된 경우
elif self.check_del_video(video_count):
    if video_count - self.youtubeData.loc[self.youtubeChannelID,
"videoCount"] < 3:
        self.youtubeData.loc[self.youtubeChannelID,
"videoCount"] -= 1
```



```

        # 데이터 저장
        asyncio.create_task(saveYoutubeData(self.youtubeData,
self.youtubeChannelID))

```

부록 5.2 유튜브 알림 전송

```

# 새 비디오 알림을 전송하는 함수
async def post_youtube(self):
    if self.new_video_json_data_list:
        self.youtubeData.loc[self.youtubeChannelID,
"video_count_check"] = 0

        # 오래된 비디오부터 처리 (역순으로 처리)
        for json_data in reversed(self.new_video_json_data_list):
            if json_data is not None:
                # 알림을 보낼 웹훅 URL 목록 가져오기
                list_of_urls= get_list_of_urls(self.DO_TEST,
self.userStateData, self.youtubechannelName, self.channel_id, "유튜브 알림")

                # 푸시 알림 및 디스코드 웹훅 전송

                asyncio.create_task(send_push_notification(list_of_urls, json_data))

                await self.DiscordWebhookSender_class.send_messages(list_of_urls, json_data)
                await asyncio.sleep(0.5) # 웹훅 전송 간 딜레이

        # 데이터 저장
        asyncio.create_task(saveYoutubeData(self.youtubeData,
self.youtubeChannelID))

```

부록 6.1 카페 데이터 확인 로직

```
async def start(self):
    self.message_list: list = []                # 알림을 보낼 게시글
    목록
    await self.getCafeDataDic()                 # 카페 데이터 가져
    오기
    await self.postCafe()                       # 카페 게시글 알림
    전송

# 카페 API에서 게시글 데이터 가져오기
async def getCafeDataDic(self):
    # 네이버 카페 API URL 구성
    BASE_URL =
    f"https://apis.naver.com/cafe-web/cafe2/ArticleListV2dot1.json,{str(self.cafe
    Data.loc[self.channel_id, 'cafeNum'])}"
    response = await get_message(self.performance_manager, "cafe",
    BASE_URL) # API 요청

    # 현재 저장된 카페 데이터 가져오기
    cafe_json = self.cafeData.loc[self.channel_id, 'cafe_json']
    cafe_json_ref_articles = cafe_json["refArticleId"]
    # 이미 처리한 게시글 ID 목록
    max_ref_article = max(cafe_json_ref_articles) if cafe_json_ref_articles
    else 0 # 최근 게시글 ID
    update_time = int(self.cafeData.loc[self.channel_id, 'update_time'])
    # 마지막 업데이트 시간
    cafe_name_dict = self.cafeData.loc[self.channel_id, "cafeNameDict"]
    # 필터링할 작성자 닉네임 목록

    try:
```

```

        # 게시물 목록 처리
        self._process_article_list(response, cafe_name_dict, max_ref_article,
update_time, cafe_json_ref_articles)
    except Exception as e:
        asyncio.create_task(log_error(f"게시글 처리 중 오류 발생: {e}"))

    return

```

부록 6.2 카페 게시물 목록 처리 로직

```

# 게시물 목록 처리 함수
def _process_article_list(self, request, cafe_name_dict, max_ref_article,
update_time, cafe_json_ref_articles):
    """
    Args:
        request: API 요청 결과
        cafe_name_dict: 필터링할 작성자 닉네임 딕셔너리
        max_ref_article: 가장 최근의 현재까지의 게시물 ID
        update_time: 마지막 업데이트 시간
        cafe_json_ref_articles: 게시물 ID 목록들
    """
    # API 응답에서 게시물 목록 추출
    article_list = request.get('message', {}).get('result', {}).get('articleList',
[])

    # 역순으로 처리 (오래된 글부터 처리)
    for article in reversed(article_list):
        #특정 작성자의 게시물만 처리
        if article["writerNickname"] not in cafe_name_dict:
            continue

```

```

        # 새 게시글인지 확인 (ID가 더 크고, 작성 시간이 더 최신)
        if not (article["refArticleId"] > max_ref_article and
article['writeDateTimestamp'] > update_time):
            continue

        # 게시글 제목 처리(특수문자 등 처리)
        article["subject"] = subjectReplace(article["subject"])

        # 최신 업데이트 시간 갱신
        self.cafeData.loc[self.channel_id, 'update_time'] = max(
            self.cafeData.loc[self.channel_id, 'update_time'],
            article["writeDateTimestamp"]
        )

        # 참조 게시글 ID 목록 업데이트 (최대 10개 유지)
        self._update_ref_article_ids(cafe_json_ref_articles,
article["refArticleId"])

        # 카페 게시글 데이터 객체 생성 및 추가
        self._add_cafe_post(article)

```

부록 6.3 카페 게시글 알림 전송

```

# 카페 게시글 알림을 전송하는 함수
async def postCafe(self):
    # 메시지 목록이 비어있으면 종료
    if not self.message_list:
        return

    # 각 게시글에 대해 알림 전송
    for post_data in self.message_list:

```

```

# 웹훅 JSON 데이터 생성
json_data = await self.create_cafe_json(post_data)
print(f"{datetime.now()} {post_data.writer_nickname} post cafe
{post_data.subject}")

# 알림을 보낼 웹훅 URL 목록 가져오기
list_of_urls = get_list_of_urls(self.DO_TEST, self.userStateData,
post_data.writer_nickname, self.channel_id, "cafe_user_json")

# 푸시 알림 및 디스코드 웹훅 전송
asyncio.create_task(send_push_notification(list_of_urls, json_data))

asyncio.create_task(self.DiscordWebhookSender_class.send_messages(list_of_
urls, json_data))

# 카페 데이터 저장
asyncio.create_task(saveCafeData(self.cafeData, self.channel_id))

```

부록 7.1 데이터베이스 스키마(userStateData)

```

create table public."userStateData" (
    "discordURL" text not null,
    username text null default 'anonymous'::text,
    뱅온 알림 text null,
    방제 변경 알림 text null,
    방중 알림 text null,
    cafe_user_json jsonb null default '{}'::jsonb,
    chat_user_json jsonb null default '{}'::jsonb,
    유튜브 알림 jsonb null default '{}'::jsonb,
    "VOD 알림" jsonb null default '{}'::jsonb,
    last_token_update text null,

```

```

fcm_tokens_data jsonb null default '[]'::jsonb,
last_db_save_time text null,
하이라이트 알림 text null,
핫클립 알림 text null,
constraint userStateData_pkey primary key ("discordURL")
) TABLESPACE pg_default;

```

부록 7.2 알림 필터링 함수

```

# 사용자 설정에 따른 전송 대상 웹훅 URL 목록 생성 함수
def get_list_of_urls(userStateData, name, channel_id, db_name):
    # 결과 URL 목록
    result_urls = []
    # 사용자 데이터에서 적절한 웹훅 URL 찾기
    for discordWebhookURL in userStateData['discordURL']:
        try:
            # 해당 URL의 사용자 데이터 가져오기
            user_data = userStateData.loc[discordWebhookURL, db_name]

            # 데이터 유형 확인 후 리스트 변환
            if isinstance(user_data, str):
                name_list = user_data # 문자열인 경우
            elif isinstance(user_data, dict):
                name_list = user_data.get(channel_id, []) # 딕셔너리인 경
우
            else:
                name_list = [] # 그 외 경우

            # 해당 이름이 리스트에 있으면 URL 추가
            if name in name_list:
                result_urls.append(discordWebhookURL)

```

```

except (KeyError, AttributeError) as e:
    # 특정 URL 처리 중 오류가 발생해도 다른 URL 처리는 계속
    continue

return result_urls

```

부록 8.1 여러 디스코드 웹훅 URL에 메시지 전송 함수

```

# 여러 웹훅 URL에 메시지를 전송하는 함수
async def send_messages(self, messages: List[str], json_data):
    # 동시 요청 제한을 위한 세마포어
    semaphore = asyncio.Semaphore(self.MAX_CONCURRENT)

    # HTTP 클라이언트 세션 생성
    async with ClientSession(connector=TCPConnector(ssl=False)) as
session:
        # 각 URL에 대한 전송 태스크 생성
        tasks = [
            asyncio.create_task(
                self._send_message_with_retry(session, url, json_data,
semaphore)
            )
            for url in messages
        ]

        # 태스크 완료 시 응답 수집
        responses = []
        for task in asyncio.as_completed(tasks):
            result = await task
            if result is not None:

```

```

        responses.append(result)

    return responses

```

부록 8.2 메시지 전송 함수

```

# 재시도 로직이 포함된 메시지 전송 함수
async def _send_message_with_retry(self,
                                     session: ClientSession,
                                     url: str,
                                     data: Dict[str, Any],
                                     semaphore: asyncio.Semaphore):

    async with semaphore: # 세마포어로 동시 요청 제한
        # 웹훅 POST 요청 전송
        async with session.post(url, json=data, timeout=self.TIMEOUT) as
response:
            response.raise_for_status() # HTTP 오류 발생 시 예외 발생
            return await response.text() # 응답 반환
    ...

    return None # 모든 시도 실패 시 None 반환

```

부록 9 플랫폼별 데이터 수집

```

async def get_message(performance_manager, platform, link): config =
platform_config.get(platform)
if not config:
    return {}
headers = getDefaultHeaders()

```



```

...
# 반복적인 재시도 및 예외 처리
While retry_count < max_retries:
    try:        response        =        awaitasyncio.to_thread(get,        formatted_url,
**request_kwargs)
    If response.status_code == 200:
        return config["response_handler"](response)
    Except Exception as e: retry_count += 1
        awaitasyncio.sleep(retry_delay)
        retry_delay *= 2    return{}

```

부록 10 하이라이트 피크 점수 판단 및 새 하이라이트 구분 로직

```

async def change_score_to_peak(self, highlight: StreamHighlight):
    if not self.highlights:
        return

    is_higher_score = False
    if highlight.fun_score > self.highlights[-1].fun_score:
        is_higher_score = True

    if (highlight.score_details['highlights'] and not
highlight.score_details['should_create_new_highlight']):
        idx = None
        is_new_highlight_check_cnt = 0
        for i,detailed_log in enumerate(reversed(self.detailed_logs)):
            # 현 사점과 직전의 하이라이트 사이에 하이라이트가 아닌 구
간이 있는지

```

```

        if not detailed_log['score_components']['highlights']:
            is_new_highlight_check_cnt += 1

        i                                                                    f
detailed_log['score_components']['should_create_new_highlight']:
        idx = i
        break

        #직전 하이라이트의 should_create_new_highlight를 False로 변경
        후 현재 것을 True로 변경
        if idx:
            # 현 사점과 직전의 하이라이트 사이에 하이라이트가 아닌 구
            간이 있다면, 직전의 하이라이트 제거하지 않고, 새로운 하이라이트 추가
            if not is_higher_score and is_new_highlight_check_cnt < 3:
                return

        highlight.score_details['should_create_new_highlight'] = True

self.detailed_logs[-1]['score_components']['should_create_new_highlight'] =
True

        if is_new_highlight_check_cnt >= 3:
            return

        highlight.comment_after_openDate =
self.detailed_logs[-(idx+1)]['comment_after_openDate']
        self.detailed_logs[-1]['comment_after_openDate'] =
self.detailed_logs[-(idx+1)]['comment_after_openDate']

self.detailed_logs[-(idx+1)]['score_components']['should_create_new_highligh
t'] = False

```

```
self.highlights = self.highlights[:-1]  
return
```

※ 전체 구현 코드는 GitHub 를 통해 열람 가능하다. 김성재[10]

2. 용어 해설

API (Application Programming Interface): 응용 프로그램에서 다른 프로그램의 기능을 사용할 수 있도록 제공하는 인터페이스. 본 연구에서는 Chzzk, Soop, YouTube, 네이버 카페의 공개 API를 활용하여 방송 및 콘텐츠 데이터를 수집한다.

VOD (Video On Demand): 시청자가 원하는 시간에 재생할 수 있도록 저장된 영상 콘텐츠. 실시간 방송이 종료된 후 플랫폼에서 제공하는 다시보기 영상을 의미한다.

FCM (Firebase Cloud Messaging): Google에서 제공하는 크로스 플랫폼 메시징 솔루션. 본 연구에서는 모바일 앱 사용자에게 푸시 알림을 전송하는 데 활용한다.

Discord Webhook: Discord 채널에 외부 애플리케이션에서 메시지를 자동으로 전송할 수 있도록 제공되는 고유 URL. 본 시스템의 주요 알림 채널로 사용된다.

Fun Score: 본 연구에서 정의한 방송의 재미도를 정량화한 지표. 채팅 급증(45%), 반응 강도(30%), 참여 다양성(10%), 시청자 추세(15%)를 종합하여 0~100점 범위로 산출한다.

하이라이트 (Highlight): 방송 중 시청자 반응이 집중되어 재미도 점수가 급상승한 구간. 본 연구에서는 score_difference가 15 이상인 구간을 하이라이트로 정의한다.

핫클립 (Hot Clip): 방송 종료 후 생성된 클립 중 조회수가 급증하거나 최신성이 높은 클립. HotScore 70점 이상인 클립을 핫클립으로 분류한다.

뱅온: 방송 시작을 의미하는 인터넷 방송 용어 (Broadcasting ON의 줄임말).

뱅종: 방송 종료를 의미하는 인터넷 방송 용어 (Broadcasting END의 줄임말).

방제: 방송 제목의 줄임말. 스트리머가 설정한 해당 방송의 제목을 의미한다.

Chzzk: 네이버에서 2023년 12월 출시한 실시간 스트리밍 플랫폼.

Soop: 아프리카TV가 2024년 리브랜딩하면서 변경한 새로운 서비스 명칭.

Embed: Discord에서 제공하는 구조화된 메시지 포맷. 제목, 설명, 이미지, 필드 등을 포함하여 시각적으로 풍부한 알림을 표현할 수 있다.

세마포어 (Semaphore): 동시에 실행 가능한 작업의 수를 제한하는 동기화 기법. 본 연구에서는 API 호출 및 알림 전송 시 서버 과부하를 방지하기 위해 사용한다.

비동기 처리 (Asynchronous Processing): 작업의 완료를 기다리지 않고 다른 작업을 동시에 수행하는 프로그래밍 기법. Python의 asyncio 라이브러리를 활용하여 구현한다.

윈도우 (Window): 데이터 스트림 분석에서 특정 시간 범위의 데이터를 하나의 단위로 처리하는 구간. 본 연구에서는 30초 윈도우를 사용하여 채팅 데이터를 분석한다.

백오프 (Backoff): 요청 실패 시 재시도 간격을 점진적으로 늘리는 기법. 본 연구에서는 지수적 백오프(exponential backoff)를 적용하여 API 요청의 안정성을 확보한다.

시그모이드 함수 (Sigmoid Function): 입력값을 0과 1 사이로 변환하는 S자 형태의 함수. 본 연구에서는 채팅 급증 및 반응 강도를 정규화하는 데 사용한다.

3. API 호출 주기 설정

Main_loop 함수에서 방송 상태 확인, VOD 상태 확인, 카페 상태 확인을 한다.

Main_loop 함수는 1초에 1번 실행된다.

방송 상태 확인:

- Chzzk는 Main_loop 함수 loop count % 2 == 0 마다 실행
- Soop은 Main_loop 함수 loop count % 2 == 0 마다 실행

VOD 상태 확인:

- Chzzk는 Main_loop 함수 loop count % 3 == 2 마다 실행
- Soop은 Main_loop 함수 loop count % 3 == 0 마다 실행
- 카페 상태 확인: loop count % 3 == 1 마다 실행

유튜브 상태 확인: 3초에 1번 실행

핫클립 작업: 30분에 1번 실행

채팅 데이터 수집: socket으로 연결하여 각 채널이 켜진 시점부터 방송 종료

후 최대 3분 까지 수신한다. 채팅은 각 채널에서 채팅 발생 할 때마다 수신
(time out 1.0sec)

4. 시스템 설정 파일

4.1. 환경변수 설정(.env)

```
#.env
# DB API key
supabase_url
supabase_key

# youtube api key
DeveloperKey

# 각종 기능 디버깅 용 Discord WebHookURL
HighlightURL
errorPostBotURL
testPostBotURL
chat_post_url
donation_post_url
afreeca_chat_log_url
DeveloperWebhookURL

# Chzzk 로그인 키
NID_AUT
NID_SES

# Soop 로그인 키
AuthTicket
```

```
# 기본 프로필 이미지(프로필 이미지 없을 경우 대체)
default_thumbnail

# Icon URLs
CHZZK_ICON
AFREECA_ICON
SOOP_ICON
BLACK_IMG=
YOUTUBE_ICON=
CAFE_ICON


# Firebase credentials (instead of the JSON file)
FIREBASE_TYPE=service_account
FIREBASE_PROJECT_ID=streamalert-a07d2
FIREBASE_PRIVATE_KEY_ID=590d5616c7caec9e5e8d315ccbaa7f1792815d5a
FIREBASE_PRIVATE_KEY
FIREBASE_CLIENT_EMAIL
FIREBASE_CLIENT_ID
FIREBASE_AUTH_URI
FIREBASE_TOKEN_URI
FIREBASE_AUTH_PROVIDER_X509_CERT_URL
FIREBASE_CLIENT_X509_CERT_URL
FIREBASE_UNIVERSE_DOMAIN


# Imgbb API 설정
IMGBB_API_KEY
# 서버 URL, ip 주소
BASE_URL
DEBUG_BASE_URL
#gemini api key
GOOGLE_API_KEY
#개발자 ID
```

MY_afreeca_ID=kimboxu

4.2. 데이터 베이스 스키마 전체 구조

```
# Soop(구 afreeca TV) 스트리머 계정 정보
create table public."afreecaIDList" (
    "channelID" text not null,
    "channelName" text null,
    profile_image text null,
    channel_color bigint null default '0'::bigint,
    "offLine_thumbnail" text null,
    "afreecaID" text null,
    constraint afreecaIDList_pkey primary key ("channelID")
) TABLESPACE pg_default;

# Chzzk 스트리머 계정 정보
create table public."chzzkIDList" (
    "channelID" text not null,
    "channelName" text null,
    profile_image text null,
    channel_color bigint null default '0'::bigint,
    channel_code text null,
    "offLine_thumbnail" text null,
    constraint chzzkIDList_pkey primary key ("channelID")
) TABLESPACE pg_default;

# Soop 채팅 필터 인원 목록
create table public."afreeca_chatFilter" (
    "channelID" text not null,
    "channelName" text null default '익명'::text,
    constraint afreeca_chatFilter_pkey primary key ("channelID")
) TABLESPACE pg_default;
```


Chzzk 채팅 필터 인원 목록

```
create table public."chzzk_chatFilter" (  
    "channelName" text not null,  
    constraint test_pkey primary key ("channelName")  
) TABLESPACE pg_default;
```

Soop 스트리머 방송 상태 정보

```
create table public."afreeca_titleData" (  
    "channelID" text not null,  
    live_state text null default 'CLOSE'::text,  
    title1 text null,  
    title2 text null,  
    update_time text null default '2025-01-01T00:00:00'::text,  
    "chatChannelId" text null,  
    "oldChatChannelId" text null,  
    state_update_time jsonb null default '{"openDate": "2025-01-01 00:00:00",  
"closeDate": "2025-01-01 00:00:00", "titleChangeDate": "2025-01-01  
00:00:00", "changeChatChannelIdDate": "2025-01-01 00:00:00"}'::jsonb,  
    constraint afreeca_titleData_pkey primary key ("channelID")  
) TABLESPACE pg_default;
```

Chzzk 스트리머 방송 상태 정보

```
create table public."chzzk_titleData" (  
    "channelID" text not null,  
    live_state text null default 'CLOSE'::text,  
    title1 text null,  
    update_time text null default '2025-01-01T00:00:00'::text,  
    "chatChannelId" text null,  
    title2 text null,
```

```

"oldChatChannelId" text null,
state_update_time jsonb null default '{"openDate": "2025-01-01 00:00:00",
"closeDate": "2025-01-01 00:00:00", "titleChangeDate": "2025-01-01
00:00:00", "changeChatChannelIdDate": "2025-01-01 00:00:00"}'::jsonb,
constraint chzzk_titleData_pkey primary key ("channelID")
) TABLESPACE pg_default;

```

VOD 상태 정보

```

create table public.video_data (
"channelID" text not null,
"VOD_json" jsonb null default '{"publishDate": "2025-01-01T00:00:00",
"videoNo_list": []}'::jsonb,
platform_name text null default ''::text,
constraint video_data_pkey primary key ("channelID")
) TABLESPACE pg_default;

```

Youtube 상태 정보

```

create table public."youtubeData" (
"YoutubeChannelID" text not null,
"channelName" text null,
"channelID" text null,
"channelCode" text null,
"oldVideo" jsonb null default '{"link1": "", "link2": "", "link3": "",
"link4": "", "link5": ""}'::jsonb,
"uploadTime" text null default '2025-01-01T00:00:00Z'::text,
"videoCount" bigint null default '0'::bigint,
thumbnail_link text null,
video_count_check bigint null default '0'::bigint,
constraint youtubeData_pkey primary key ("YoutubeChannelID")
) TABLESPACE pg_default;

```

네이버 카페 상태 정보

```
create table public."cafeData" (  
    "channelID" text not null,  
    "channelName" text null,  
    "cafeID" text null,  
    "cafeNum" bigint null,  
    update_time bigint null,  
    cafe_json jsonb null,  
    "cafeName" text null,  
    "cafeNameDict" jsonb null,  
    constraint cafeData_pkey primary key ("channelID")  
) TABLESPACE pg_default;
```

핫클립 상태 정보

```
create table public.hot_clip_data (  
    "channelID" text not null,  
    platform_name text null,  
    last_updated text null default '2025-01-01T00:00:00.000000'::text,  
    sent_clip_uids jsonb null default '[]'::jsonb,  
    constraint afreeca_hot_clip_pkey primary key ("channelID")  
) TABLESPACE pg_default;
```

유저 상태 정보

```
create table public."userStateData" (  
    "discordURL" text not null,  
    뱅온 알림 text null,  
    방제 변경 알림 text null,  
    방중 알림 text null,  
    cafe_user_json jsonb null default '{}'::jsonb,
```

```

chat_user_json jsonb null default '{} '::jsonb,
username text null default 'anonymous'::text,
유튜브 알림 jsonb null default '{} '::jsonb,
"VOD 알림" jsonb null default '{} '::jsonb,
last_token_update text null,
fcm_tokens_data jsonb null default '[] '::jsonb,
last_db_save_time text null,
하이라이트 알림 text null,
핫클립 알림 text null,
constraint userStateData_pkey primary key ("discordURL")
) TABLESPACE pg_default;
# 서버 설정 상태 정보 및 관리
create table public.date_update (
    idx bigint generated by default as identity not null,
    user_date boolean null,
    all_date boolean null,
    chat_json jsonb null,
    is_youtube boolean null,
    is_vod_json jsonb null,
    is_vod_chat_json jsonb null,
    is_hot_clip jsonb null default '{} '::jsonb,
    save_highlight_data boolean null,
    is_save_highlight_data jsonb null,
    constraint date_update_pkey primary key (idx)
) TABLESPACE pg_default;

```